

# CYPHER

## Training School on Scientific Machine Learning for Digital Twins

*System Identification and machine  
learning for predicting reacting dynamics*

Anh Khoa Doan

Department of Aeronautics

[n.doan@imperial.ac.uk](mailto:n.doan@imperial.ac.uk)



# Acknowledgement

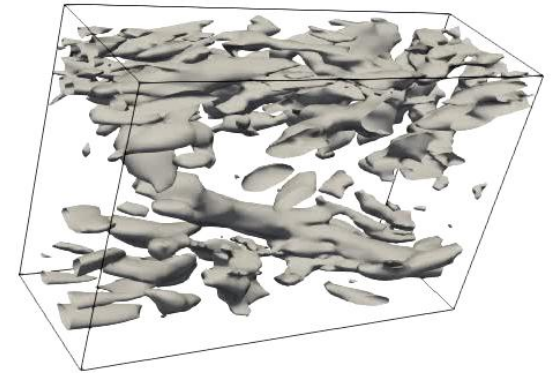
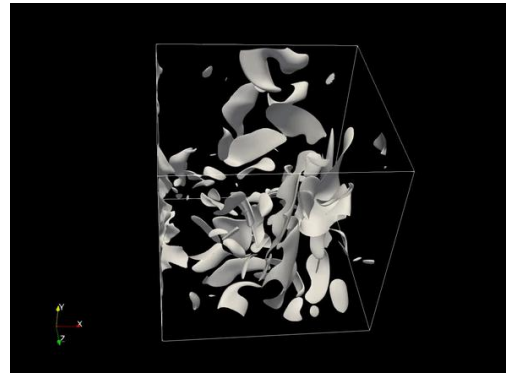
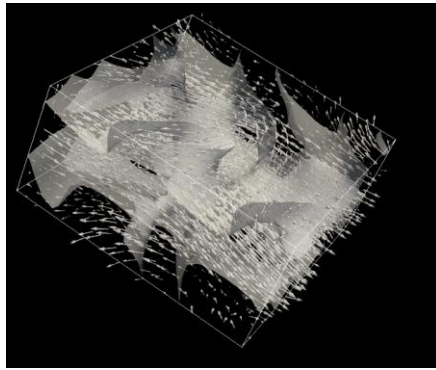
- Content heavily inspired by works/presentation of collaborators
  - Prof. Luca Magri (Imperial College):
    - Prediction of chaotic dynamics from data: An introduction (from the VKI lecture series) <https://arxiv.org/pdf/2604.11624>
    - <https://github.com/MagriLab/Tutorials>
  - Prof. Wolfgang Polifke (TU Munich):
    - Black-box system identification for reduced order model construction, *Annals of Nuclear Energy*, 67, 109-128 (2014).  
<https://doi.org/10.1016/j.anucene.2013.10.037>
    - <https://gitlab.lrz.de/tfd/system-identification-tutorial>

# Outline

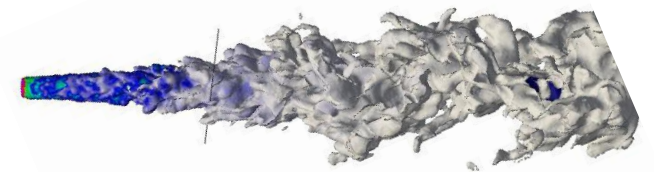
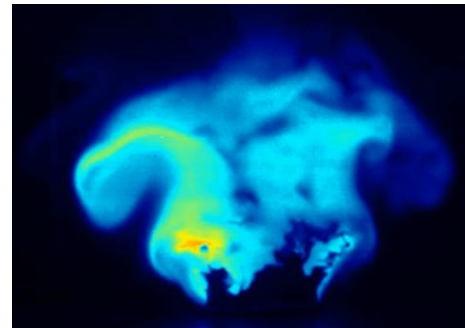
- Key Concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and dynamical systems
  - System Identification for LTI systems
  - Data-driven nonlinear dynamical system modelling
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

# Turbulent (reacting) fluid flows are difficult to predict

- Turbulence: multiscale, unsteadiness, nonlinear  
→ Unpredictable



Added chemistry complexity  
in reacting flows



# Dynamics of turbulent (reacting) flows

Nonlinear autonomous dynamical system:

$$\dot{\phi} = N(\phi, p, F)$$

$$\phi(x, 0) = \phi_0$$

$N$ : smooth, deterministic and ergodic

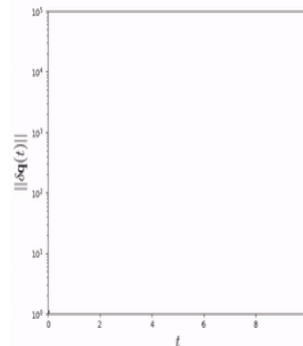
$p$ : parameters of the systems

$F$ : (possible) forcing term

For fluids:  $N$ : (Reacting) Navier-Stokes equations

Many systems of interest are *chaotic*

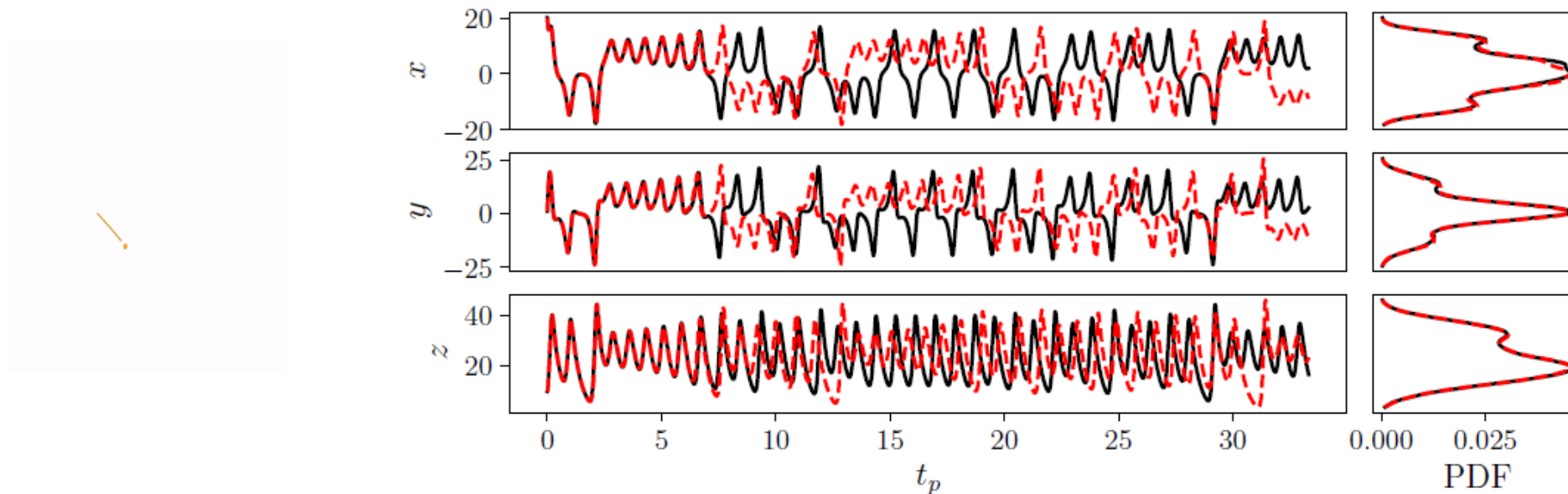
“Butterfly effect”



So what can/should we predict?

# Statistics of chaotic signals

- Statistics of chaotic signals not affected by butterfly effect  
→ Prediction of statistics becomes possible

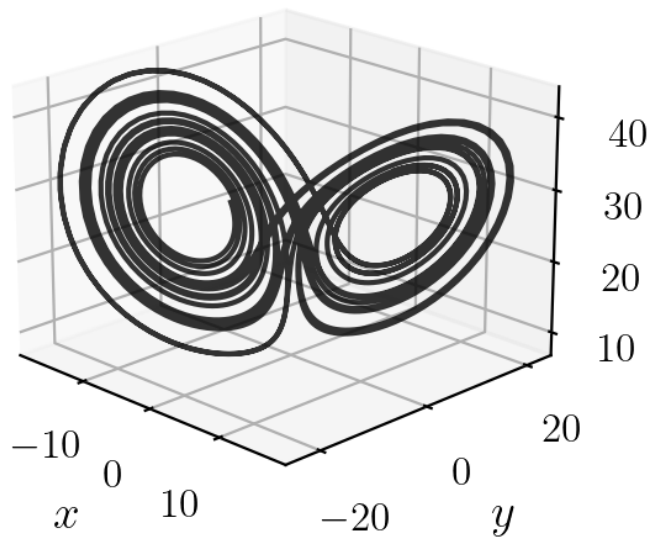


# Attractor: set of values that $q$ takes asymptotically

Nonlinear autonomous dynamical system:

$$\dot{\phi} = N(\phi, p, F)$$

$$\phi(x, 0) = \phi_0$$



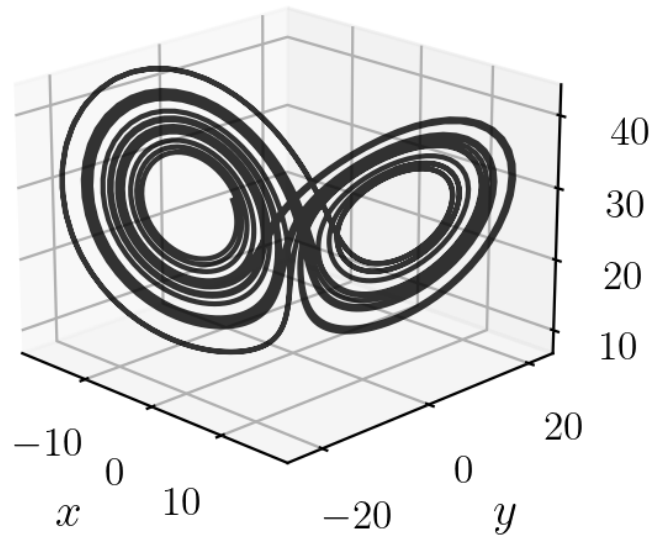
Attractor: once you are on the attractor, you remain on it

“Strange” attractor:

- Zero-measure in its embedding space
- Fractal dimension

# Some indicator of chaos

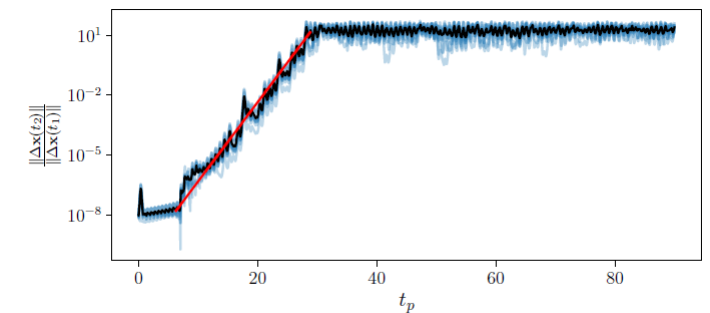
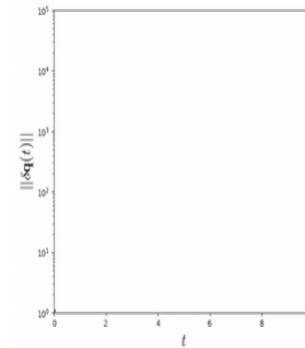
## Characteristics of attractor



Estimate of the dimension of the attractor:

- Kaplan-Yorke dimension (can be estimated using the Lyapunov spectrum, not elaborated here)

## Largest Lyapunov exponent

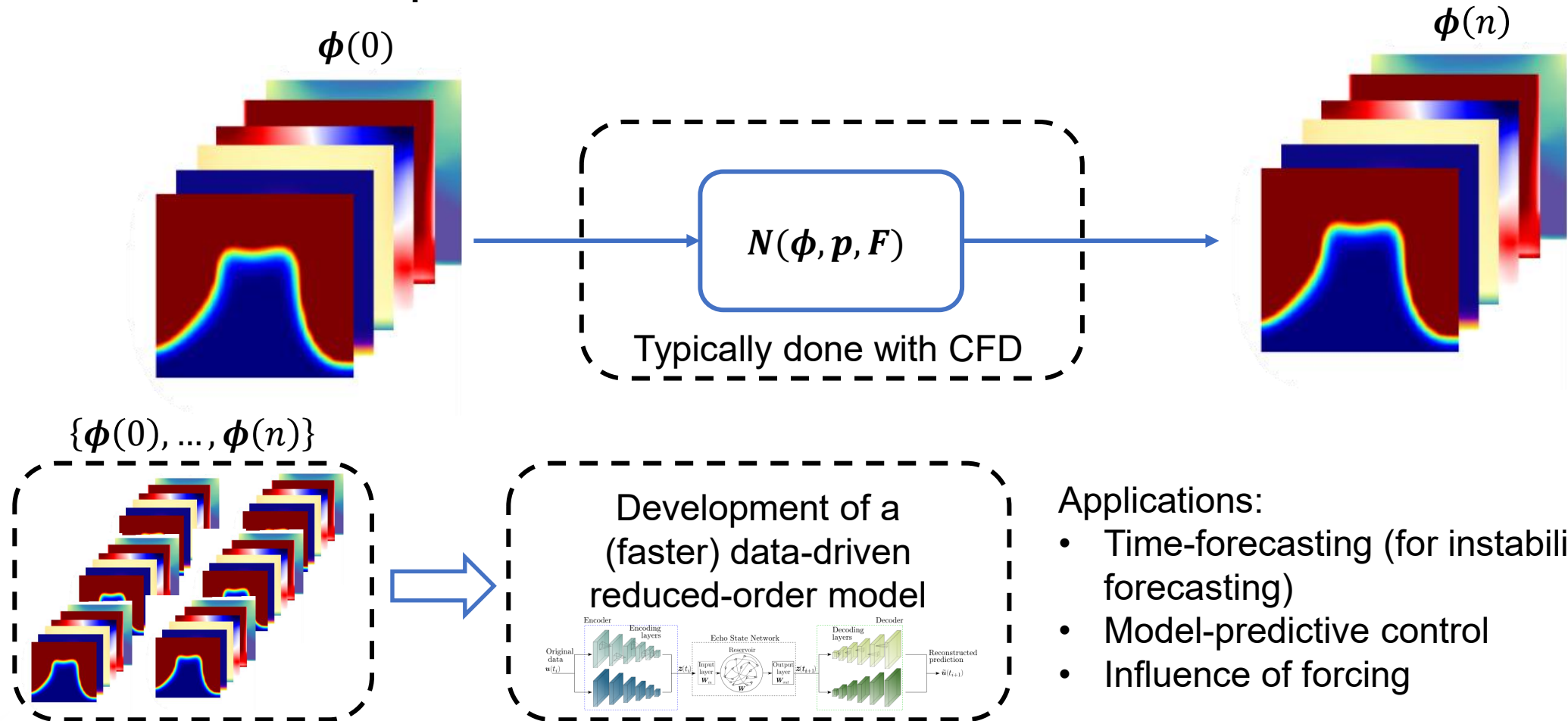


- $\lambda_1$  Lyapunov exponent: “Rate of divergence” of those two trajectories
- Lyapunov time:  $t_p = 1/\lambda_1$   
→ Indicator of “predictability” of the chaotic system



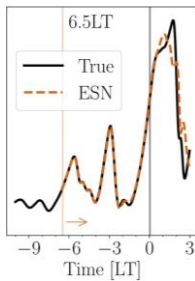
# Typical dynamical modelling in turbulent (reacting) flows: Flow state prediction

- Flow state prediction:



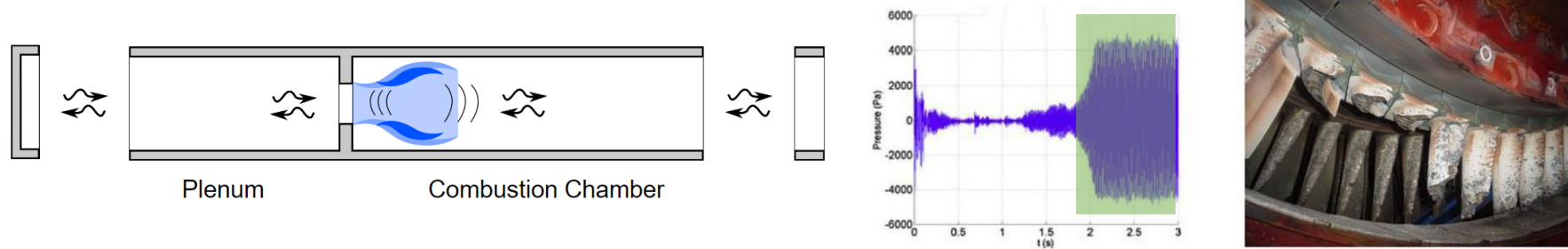
Applications:

- Time-forecasting (for instability forecasting)
- Model-predictive control
- Influence of forcing

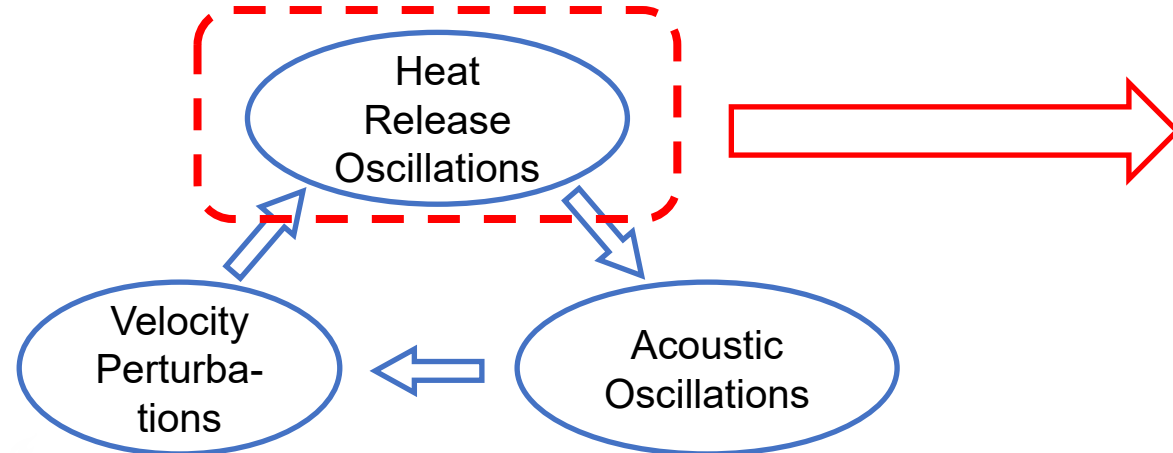


# Typical dynamical modelling in turbulent (reacting) flows: Flame dynamics modelling

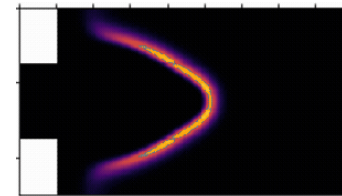
- Thermoacoustic instabilities: High amplitude pressure oscillations in premixed combustors



Thermoacoustic instabilities: result from coupling between HRR and acoustic fluctuations



$$\dot{\phi} = N(\phi, p, F)$$



$$\begin{aligned} \rightarrow \dot{Q} &= \mathcal{N}(\phi, F) \\ \rightarrow \dot{Q} &= \mathcal{F}(F_u) \end{aligned}$$

Modelling aim:

- Prediction of flow (or QoI) response to specific forcing

# Outline

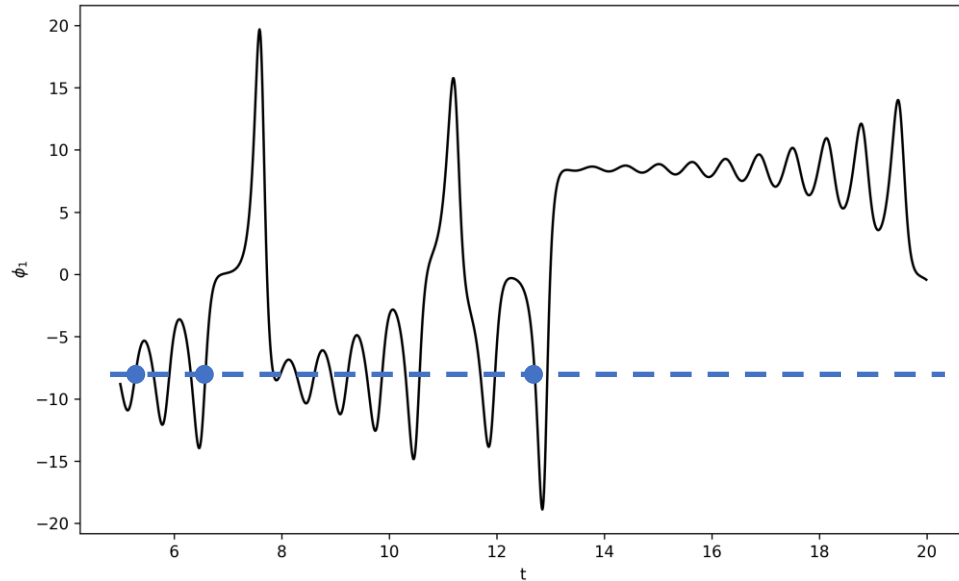
- Key Concepts in Dynamical Systems and Chaos
- **Machine Learning for Dynamical Systems**
  - Sequential data and dynamical systems
  - System Identification for LTI systems
  - Data-driven nonlinear dynamical system modelling
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

# Dynamical systems produce sequential data

Nonlinear autonomous dynamical system:

$$\dot{\phi} = N(\phi, p, F)$$

$$\phi(x, 0) = \phi_0$$



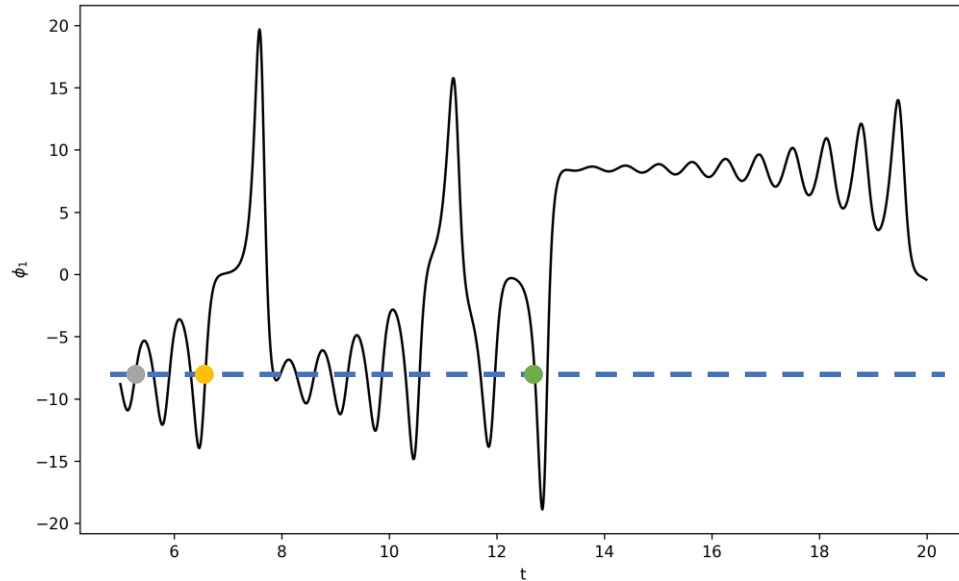
- System exhibit multiple time the same values

# Dynamical systems produce sequential data

Nonlinear autonomous dynamical system:

$$\dot{\phi} = N(\phi, p, F)$$

$$\phi(x, 0) = \phi_0$$

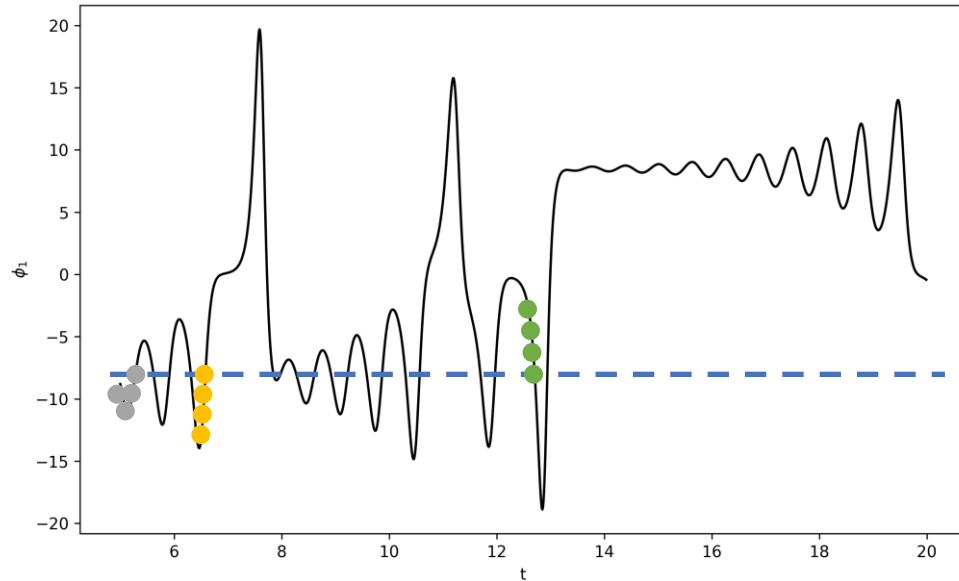


- System exhibit multiple time the same values
- But these are different data points

# Dynamical systems produce sequential data

Nonlinear autonomous dynamical system:

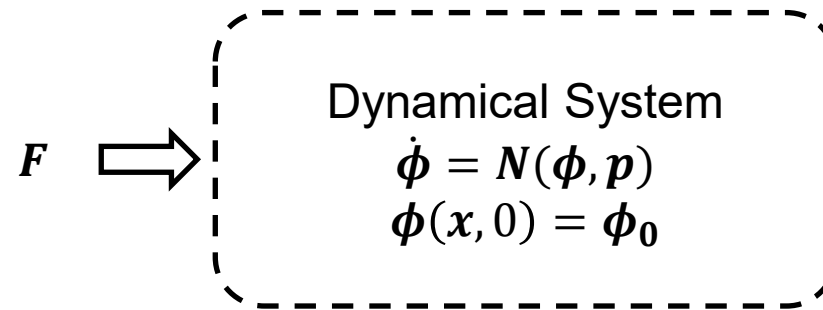
$$\dot{\phi} = N(\phi, p, F)$$
$$\phi(x, 0) = \phi_0$$



- System exhibit multiple time the same values
- But these are different data points
- The previous data provides the information
  - The *sequence* of data point is important

# Slight change in notation

- Forcing considered as an “input” to the system



- Discrete-time notation:
  - $\phi(n)$  or  $\phi_n$ : state sampled at time  $t_n$
- Objective of data-driven dynamical system modelling:
  - Based on some data of the dynamical system...
  - ...Obtain a model that can forecast  $\tilde{\phi}(n)$  (or a quantity of interest  $\tilde{Q}(\tilde{\phi})$ ) which approximates the true evolution  $\phi(n)$  (or of  $Q(\phi)$ )

# Data-driven Modelling of Dynamical Systems

- Has strong ties to “system identification”
- Has a strong history (before “machine learning”)
- Finds its root in signal processing and control theory
  - Generally developed to model systems from measured data
  - Mostly applied to linear, time-invariant systems
  - See e.g. Rabiner and Gold (1975); Bellanger (2000); Ljung (1999) for overview
- Extension to nonlinear dynamical systems
  - With recent neural networks-based approaches

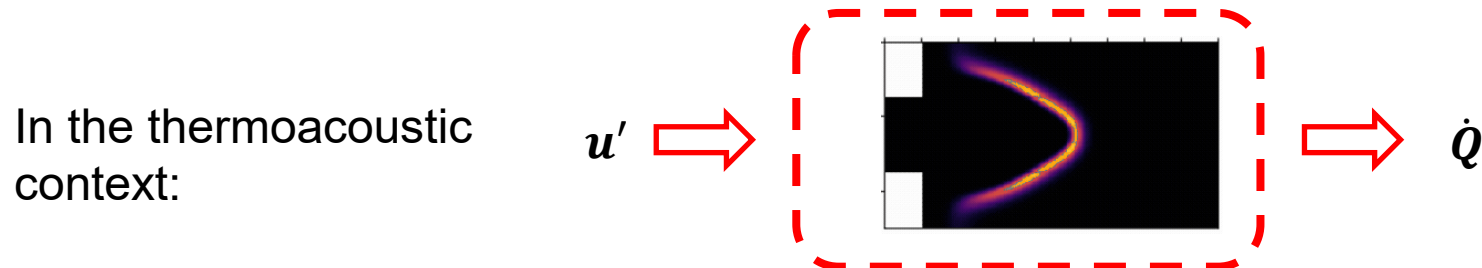
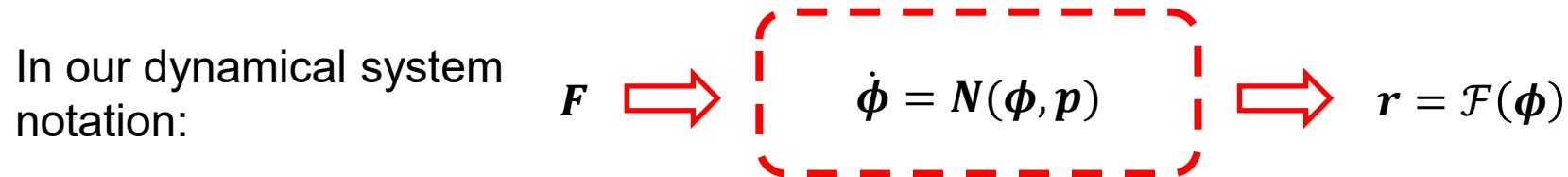
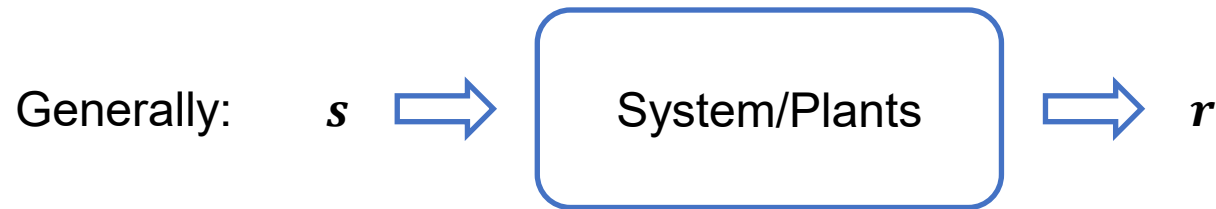


# Outline

- Key Concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and dynamical systems
  - **System Identification for LTI systems**
  - Data-driven nonlinear dynamical system modelling
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

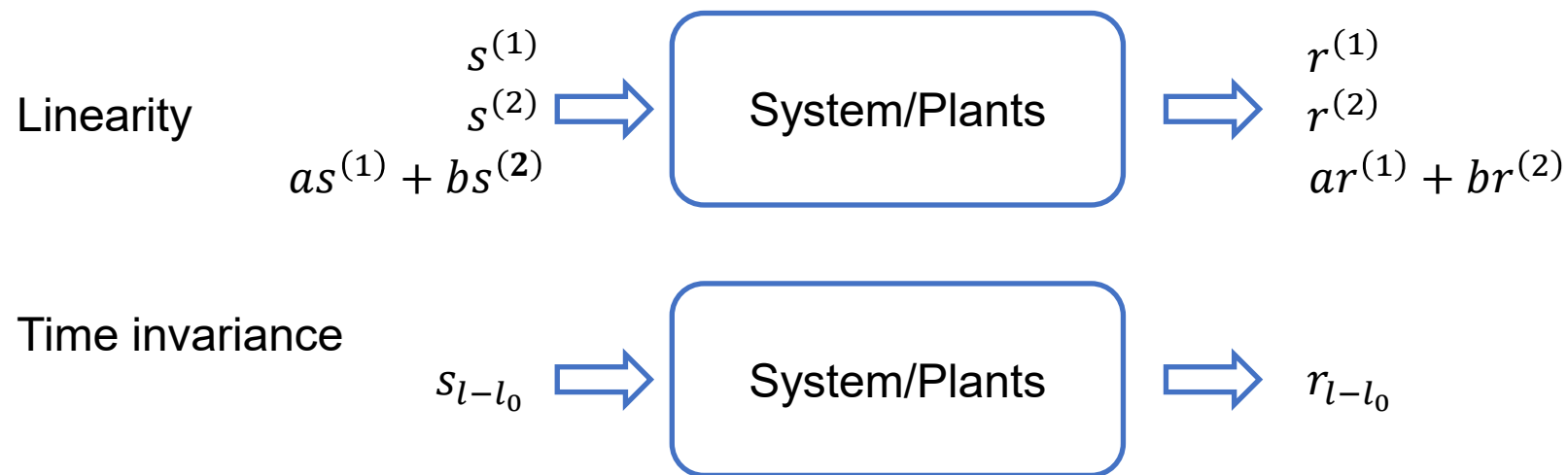
# Brief introduction to system identification

- Presentation based on review by Polifke
- Focus is on “system’s response to a signal”



# Brief introduction to system identification

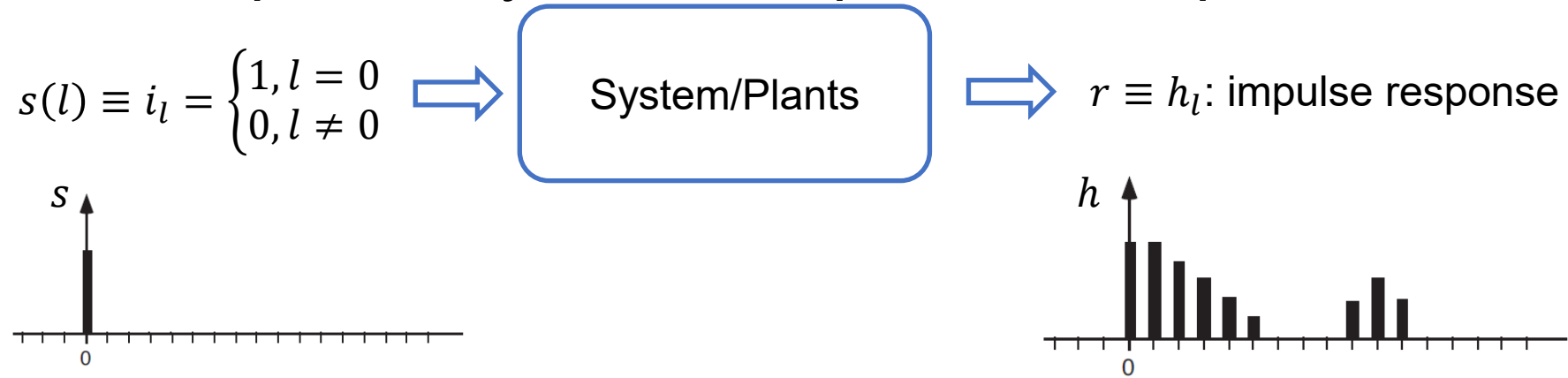
- Assumption: causal linear time-invariant (LTI) system (and for single input/output)



Causal: response  $r_l$  depends only on the present/past signal, i.e.:  
 $r_l$  does not depend on  $s_m$  for  $m > l$

# Brief introduction to system identification

- Impulse response: system's response to “impulse”



- Impulse response: characterizes the LTI system's response to *any* signal

$$r_l = \sum_{k=-\infty}^{\infty} h_k s_{l-k} \rightarrow r = h * s \quad (\text{convolution})$$

For causal system:  $h_k = 0$  for  $k < 0$

→ Fully characterizes the system's dynamics

$$r_l = \sum_{k=0}^{\infty} h_k s_{l-k}$$

# Brief introduction to system identification

- How can we estimate the impulse response  $h_k$ ?
- First, consider the finite response ( $k = 0, \dots, N$ ):

$$r_l \approx \sum_{k=0}^N h_k s_{l-k}$$

- Assume we have collected some signal/response of length  $L$ :  
 $\mathbf{s} \equiv \{s_l\}$  and  $\mathbf{r} \equiv \{r_l\}$  ( $l = 0, \dots, L$ )
- Correlation analysis can be used to estimate  $h_k$

# Brief introduction to system identification

- Consider the auto-correlation matrix  $\mathbf{\Gamma}$  of  $s$  (estimated using  $L$  time lags)

$$\Gamma_{ij} \approx \frac{1}{L - N + 1} \sum_{k=N}^L s_{k-i} s_{k-j}, \quad i, j = 0, \dots, N$$

- Cross correlation vector  $\mathbf{c}$

$$c_i \approx \frac{1}{L - N + 1} \sum_{k=N}^L s_{k-i} r_k, \quad i = 0, \dots, N$$

- One can show the following relation:

$$\begin{aligned} \mathbf{\Gamma} \mathbf{h} &= \mathbf{c} \text{ (Wiener-Hopf equation)} \\ &\rightarrow \mathbf{h} = \mathbf{\Gamma}^{-1} \mathbf{c} \end{aligned}$$

# Brief introduction to system identification

- Direct inversion of  $\mathbf{\Gamma}$  can result in “over-fitting” of the impulse response coefficients

$$\rightarrow \mathbf{h} = \mathbf{\Gamma}^{-1} \mathbf{c}$$

- Regularization generally used -  $\mathbf{h}$  such that:

$$\min(\|\mathbf{\Gamma} \mathbf{h} - \mathbf{c}\|^2 + \alpha^2 \|\mathbf{R} \mathbf{\Gamma}\|^2)$$

$\mathbf{R}$ : Regularization matrix (can be the identity matrix – Ridge regression)

Exact solution:

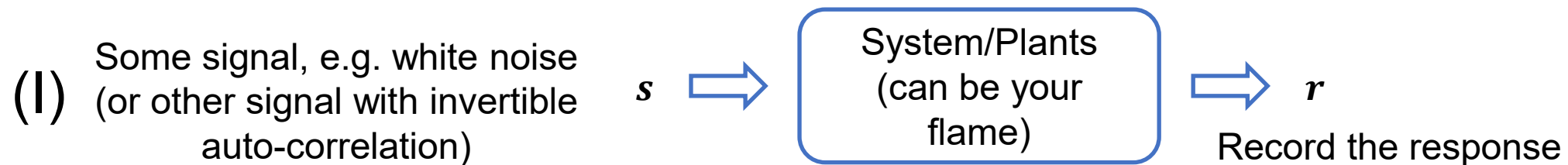
$$\rightarrow \mathbf{h}_{\alpha} = (\mathbf{\Gamma}^T \mathbf{\Gamma} + \alpha^2 \mathbf{R} \mathbf{R}^T)^{-1} \mathbf{\Gamma}^T \mathbf{c}$$

# Brief introduction to system identification

$$\rightarrow \mathbf{h} = \mathbf{\Gamma}^{-1} \mathbf{c} \text{ or } \mathbf{h}_{\alpha} = (\mathbf{\Gamma}^T \mathbf{\Gamma} + \alpha^2 \mathbf{R} \mathbf{R}^T)^{-1} \mathbf{\Gamma}^T \mathbf{c}$$

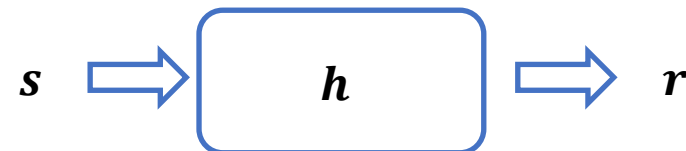
- If  $s$  is white noise,  $\mathbf{\Gamma} = \mathbf{I}$  (identity matrix)

- Summary



(II) Compute  $\mathbf{\Gamma}, \mathbf{c}$

(III) Deduce  $\mathbf{h} = \mathbf{\Gamma}^{-1} \mathbf{c}$  or  
 $\mathbf{h}_{\alpha} = (\mathbf{\Gamma}^T \mathbf{\Gamma} + \alpha^2 \mathbf{R} \mathbf{R}^T)^{-1} \mathbf{\Gamma}^T \mathbf{c}$



(IV) With  $\mathbf{h}$ , estimate  $\mathbf{r}$  for any  $\mathbf{s}$



# Brief introduction to system identification

- Approach can be extended to multi-input/multi-output systems
- In the specific case of thermo-acoustic, interest is in the frequency response:

If  $s_l(\omega) = e^{1i\omega\Delta t l}$ :

$$r_l = \sum_{k=-\infty}^{\infty} h_k e^{1i\omega\Delta t(l-k)} = e^{1i\omega\Delta t l} \sum_{k=-\infty}^{\infty} h_k e^{-1i\omega\Delta t k}$$

$$\rightarrow F(\omega) \equiv \sum_{k=-\infty}^{\infty} h_k e^{-1i\omega\Delta t k} \quad (\text{Laplace/z-transform of } \mathbf{h}, \text{ estimated at } z = e^{1i\omega\Delta t})$$

$$\rightarrow r_l(\omega) = s_l(\omega)F(\omega)$$

# Brief introduction to system identification

- Practical considerations:
  - Turbulent fluctuations can mask “noise” the response signal, so masking/filtering of turbulent fluctuation may be necessary
  - “Optimal” excitation signal  $s$  (for acoustic identification):
    - Excitation should be uncorrelated with itself
    - Signal should be broadband, low-pass filtered with constant amplitude in frequency range of interest
- ...

# Outline

- Key Concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and dynamical systems
  - System Identification for LTI systems
  - **Data-driven nonlinear dynamical system modelling**
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

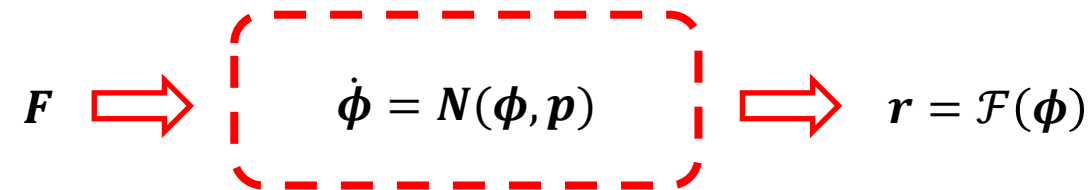
# Outline for

## *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
2. Backpropagation through Time
3. Long Short-Term Memory Unit
4. Other RNNs/Advanced RNNs

# Motivation

- So far: system identification for LTI system:



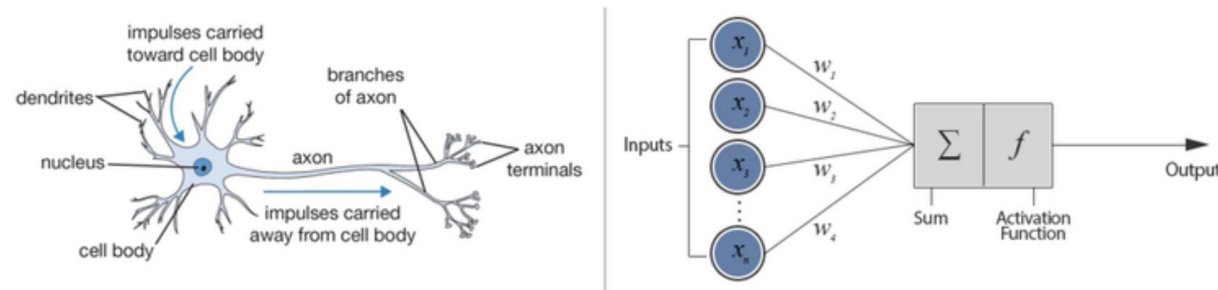
Strong assumptions remain:

→ Linear dynamical system

- What about when the system is nonlinear?
  - Various approaches possible (Volterra series, autoregressive models, ...)
  - Focus here on: neural network-based nonlinear modelling

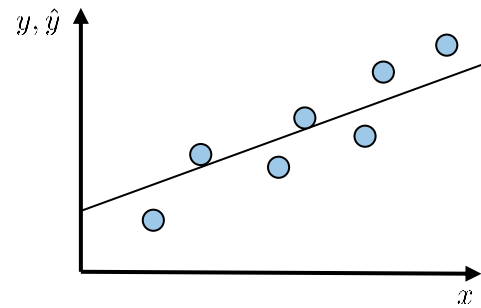
# Neural network can be used as a basis for “any model”

- Neural networks
  - Inspired to reproduce the biological process of learning (late 1940s/early 1950s)



- Universal approximator
  - “Can approximate any kind of nonlinear function”
- Very strong expressivity
  - “Can represent a large variety of functions”

# The neuron introduces nonlinearity through its activation function

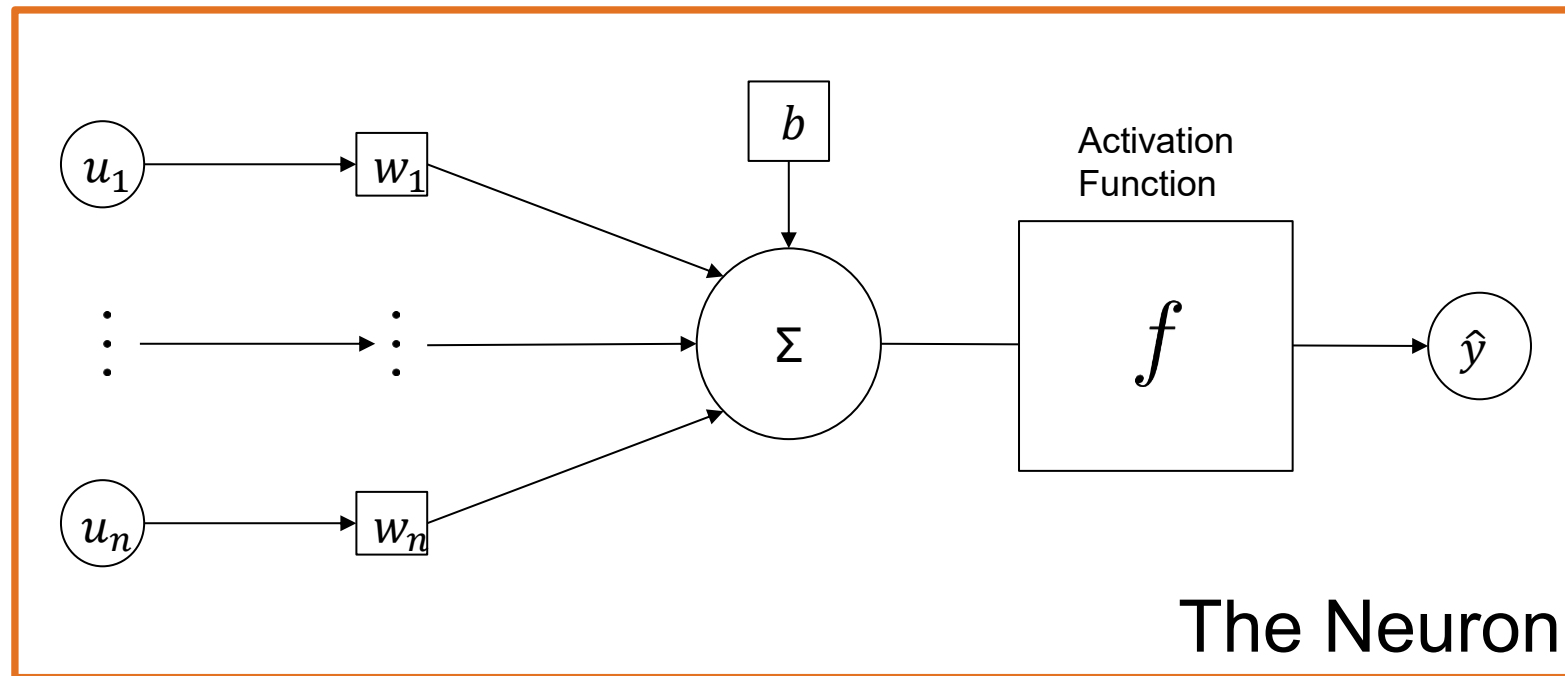


Standar linear regression:

$$\hat{y} = \sum_i^n w_i u_i + b_i = \mathbf{w} \cdot \mathbf{u} + \mathbf{b}$$

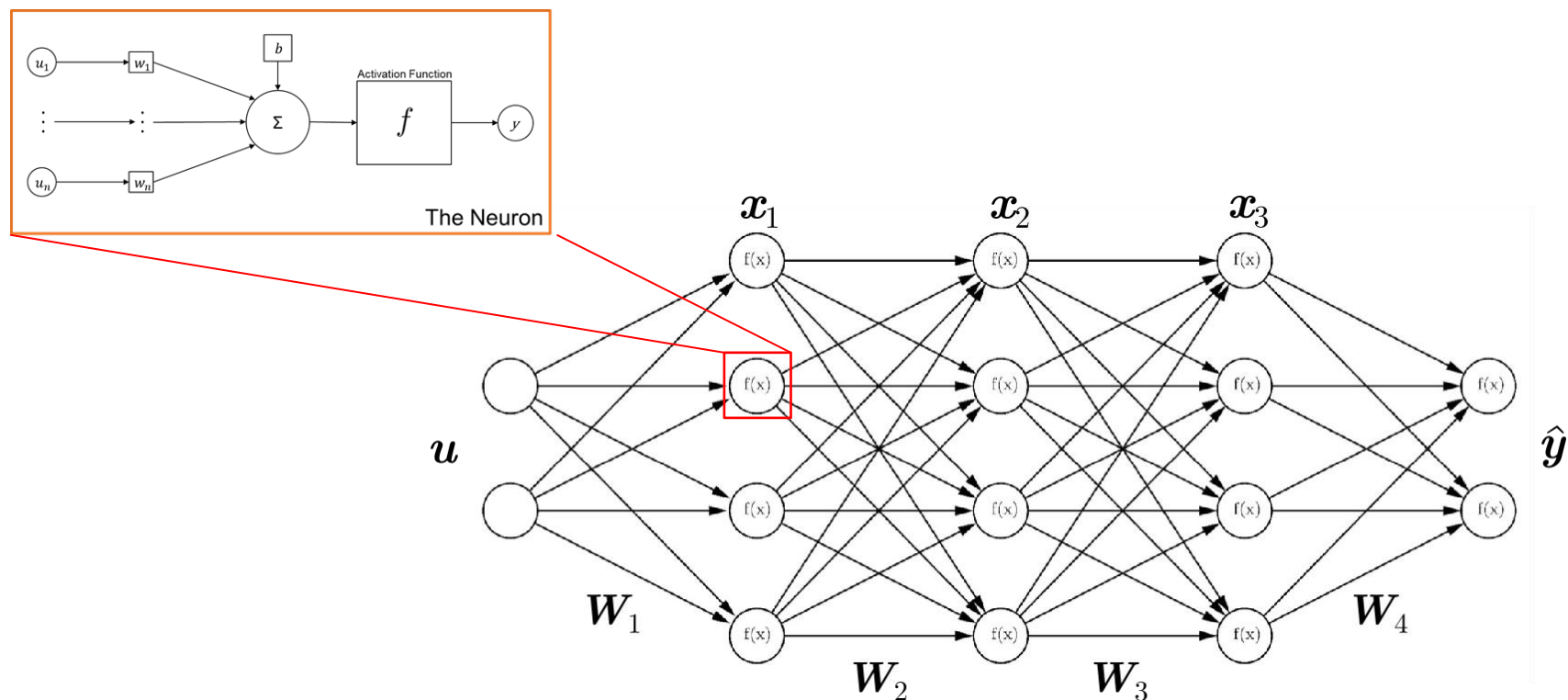
Nonlinearity introduced with activation function :

$$\hat{y} = f(\mathbf{w} \cdot \mathbf{u} + \mathbf{b})$$



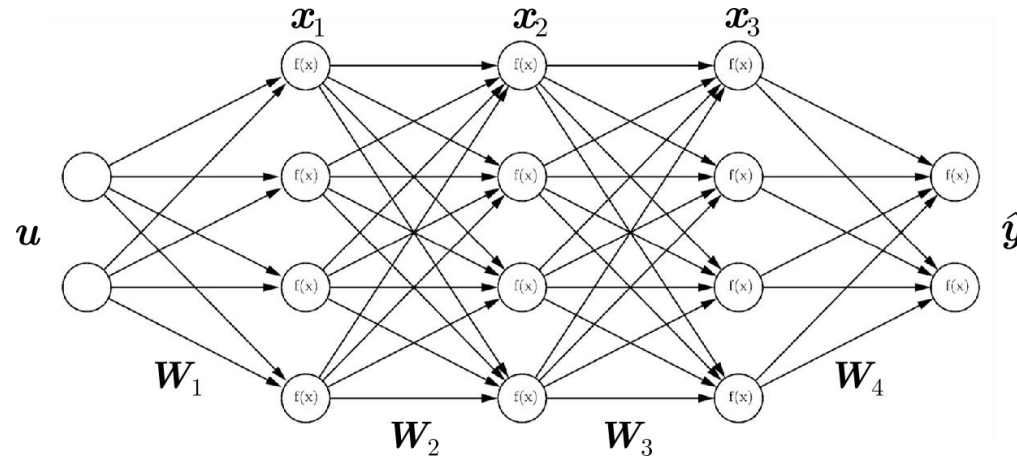
# Feedforward neural network/Multilayer perceptron are obtained by chaining layers of neurons

- Dense deep neural network:
  - Fully connected neurons organised in layers
  - Enough neurons can approximate any function (universal approximator)





# Learning/training is just optimizing the neurons' weights and biases



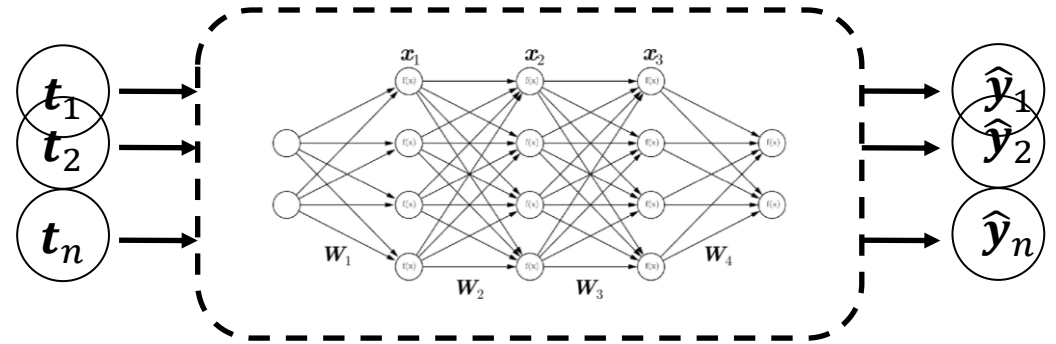
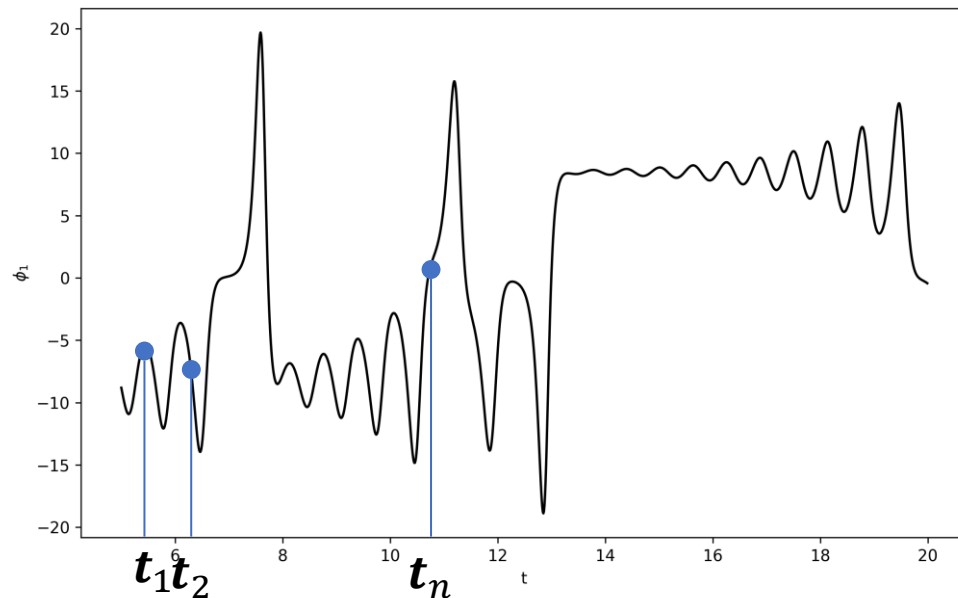
- Simplest type of task, data-regression: Minimization of the error:

$$\min_{W, b} \frac{1}{N} \sum_{i=1}^N ||y_i - \hat{y}_i||^2$$

→ Typically solved using gradient-based optimization where gradient obtained by automatic differentiation

# Naive approach to sequence processing

- How to deal with sequence?
- Possible approach: time as input

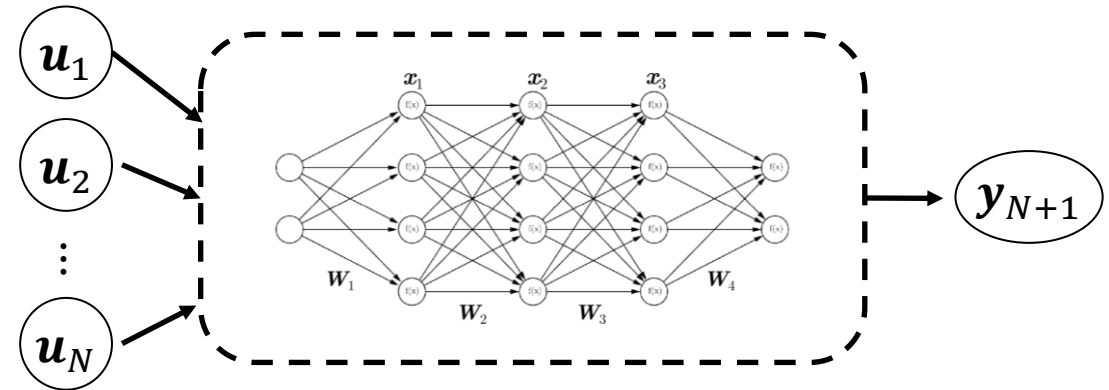
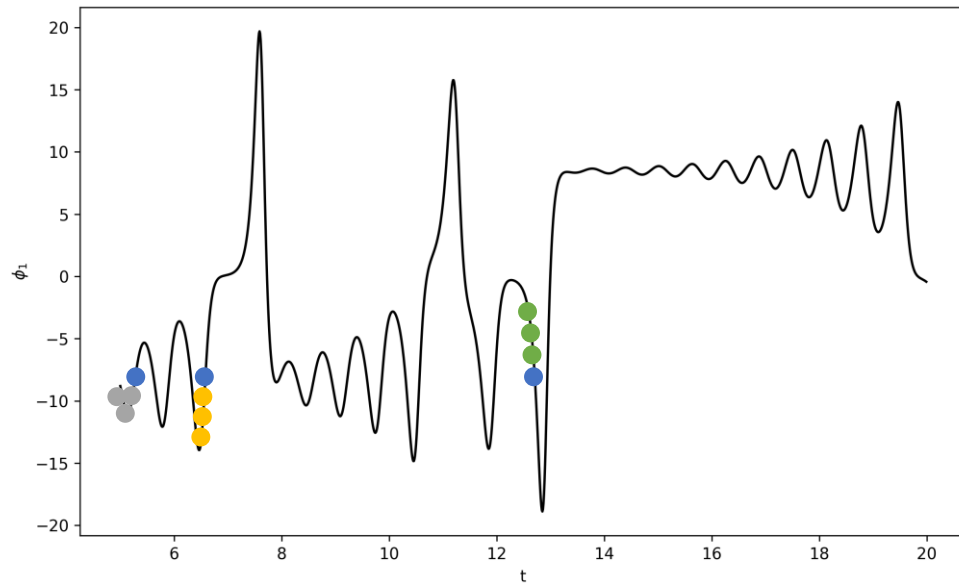


→ If input is time, no sense of “ordering”

→ Neural network: “value indexing”, no possibility to extrapolate

# Another naive approach to sequence processing

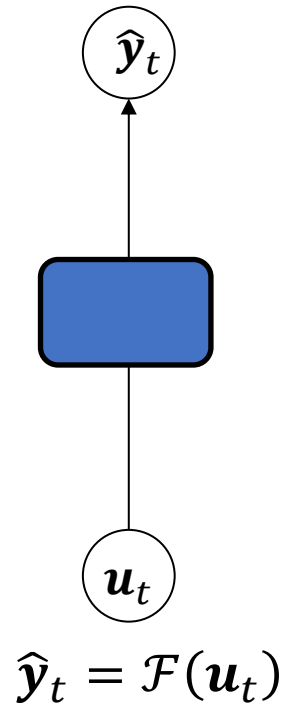
- Possible approach:



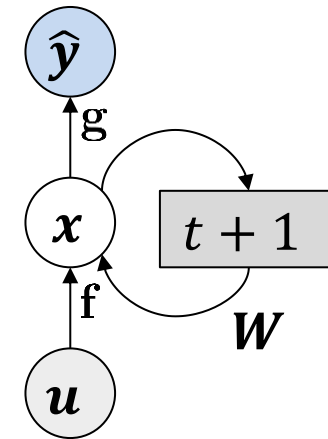
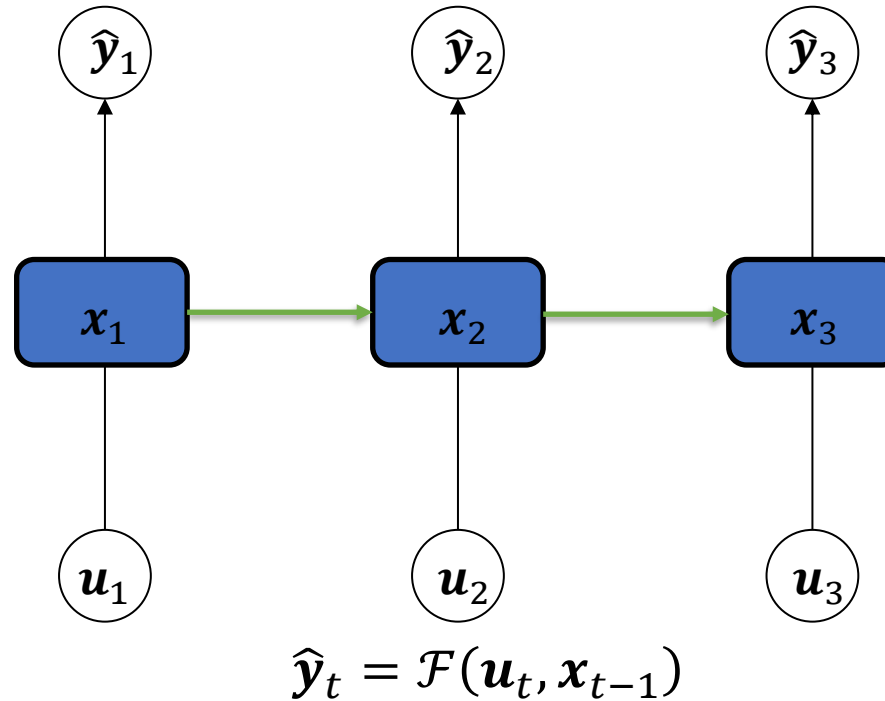
- Use sequence of  $N$  past values as input
- Requires initial guess of time-dependency
  - Can lead to large number of weights if long-term dependency required

# How to include time-dependency

Standard neuron



Time-handling neuron



$$x_t = f(x_{t-1}, u_t)$$

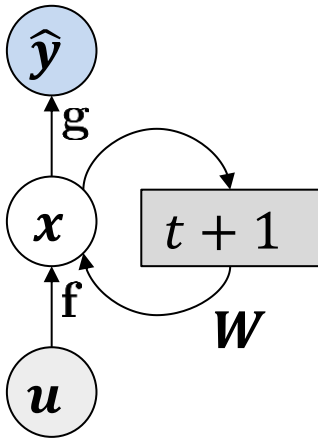
$$\hat{y}_t = g(x_t)$$

$x$ : internal/hidden state

$u$ : input

$\hat{y}$ : output

# Recurrent neural network uses a “memory” to retain past information



$$x_t = f(x_{t-1}, u_t; W)$$
$$\hat{y}_t = g(x_t; W_g)$$

$x$ : internal/hidden state  
 $u$ : input  
 $\hat{y}$ : output

## Recurrent neural network

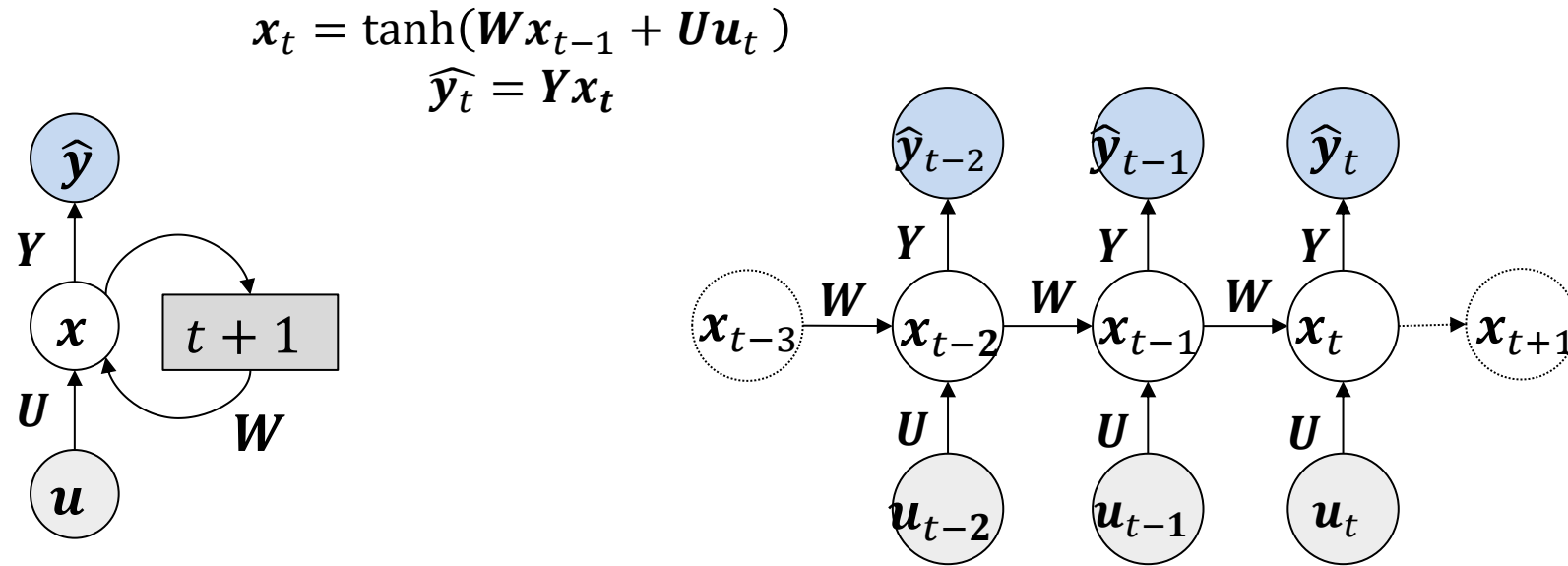
- Have a “state/memory” which is retained and evolves
- Designed to deal with time series
- Wrap up: Is a state space system that can be time-advanced whose parameters have to be learned

# Outline for

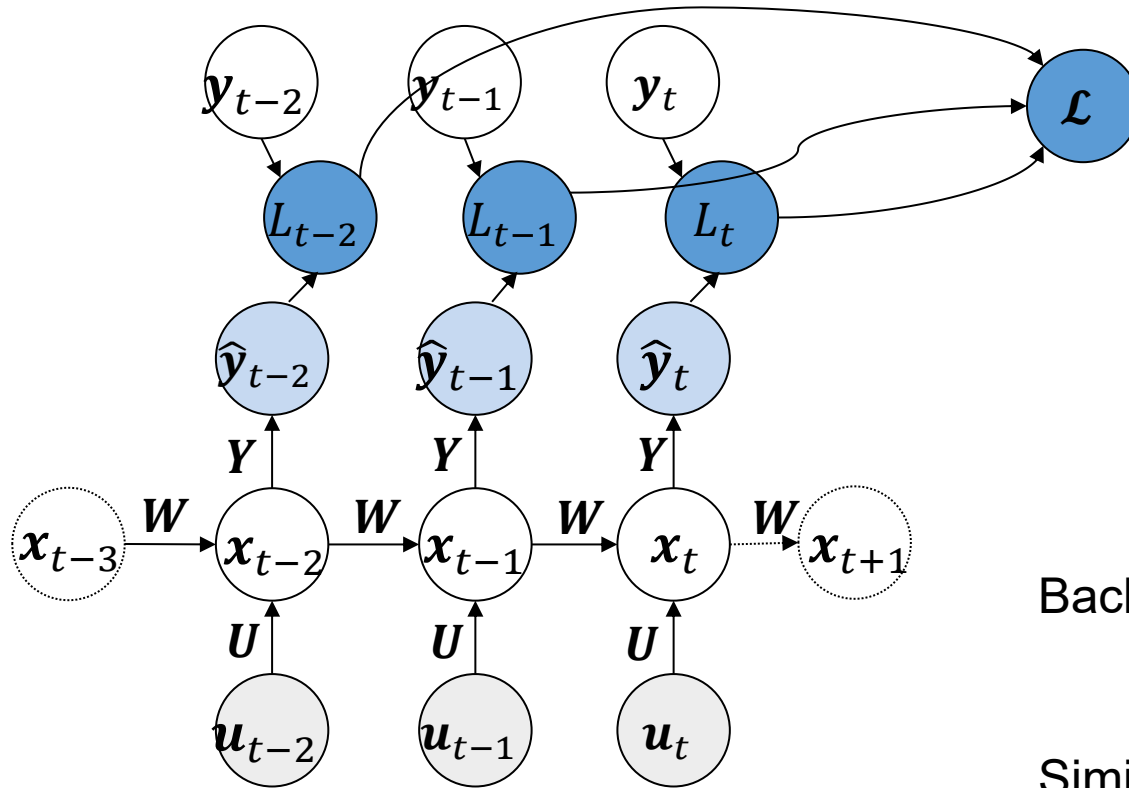
## *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
- 2. Backpropagation through Time**
3. Long Short-Term Memory Unit
4. Other RNNs/Advanced RNNs

# Backpropagation in time: Unfolding the graph



# Backpropagation in time: Unfolding the graph



Loss over multiple timesteps:

$$\mathcal{L} = \sum_{t=\tau}^t L_k$$

Backpropagation:

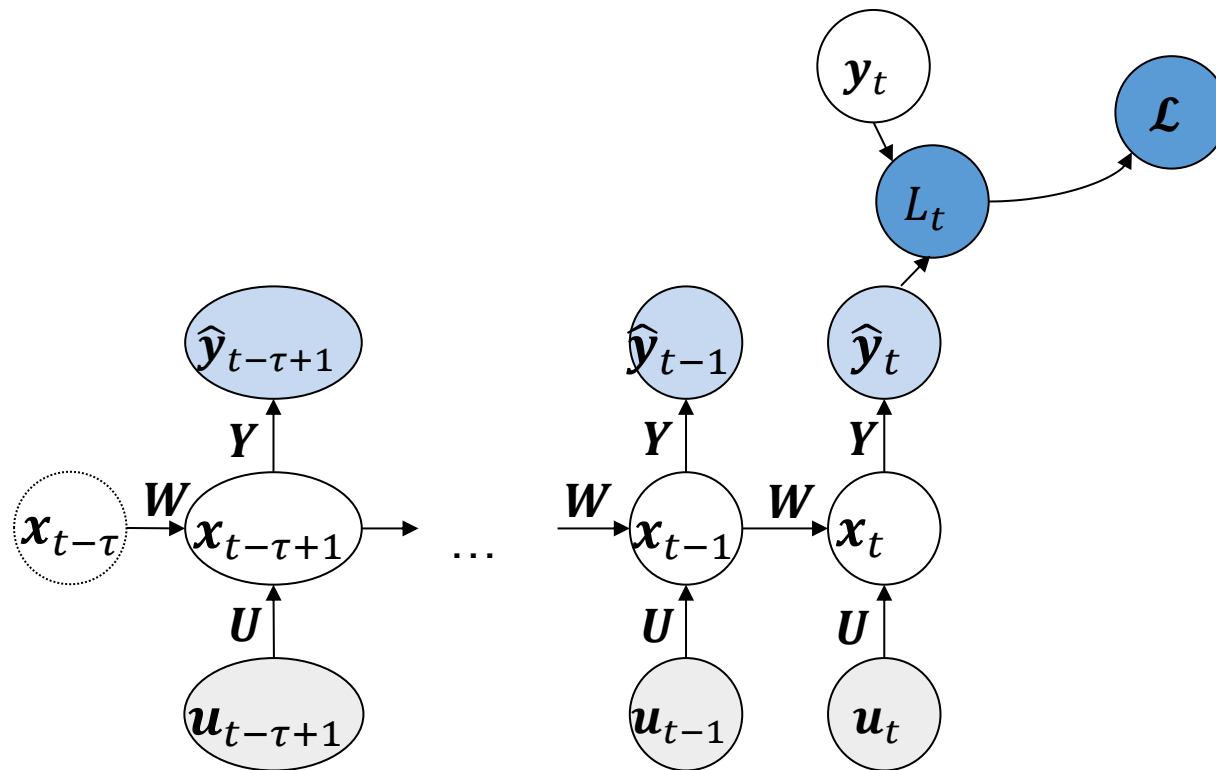
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial x_t} \frac{\partial x_t}{\partial W} + \dots + \frac{\partial \mathcal{L}}{\partial x_{t-\tau}} \frac{\partial x_{t-\tau}}{\partial W}$$

Similar to deep network:

#timesteps  $\approx$  #layers



# Long-term issues of backpropagation in time



$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{t-\tau}} = \frac{\partial \sum L_t}{\partial \mathbf{x}_{t-\tau}} \rightarrow \frac{\partial L_t}{\partial \mathbf{x}_{t-\tau}} = \frac{\partial L_t}{\partial \mathbf{x}_t} \frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_{t-\tau}}$$

Difficulty:

The gradient with respect to past goes through many applications of  $W$  and activation function if long-term dependency.

So, stability of gradient calculation depends on:

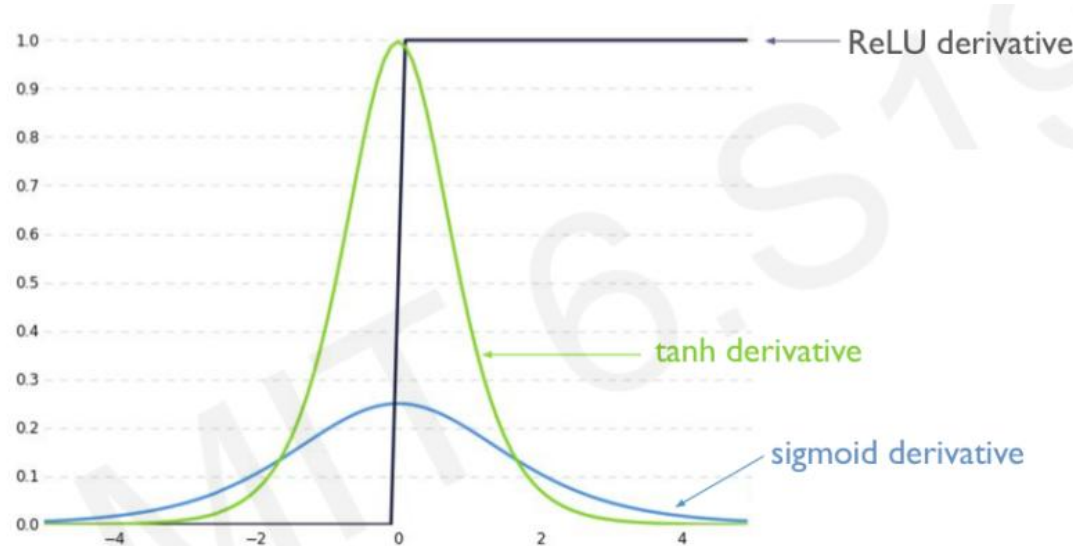
- Activation function
- Spectral radius of  $W$ .

$\rho(W) > 1$ : exploding gradient

$\rho(W) < 1$ : vanishing gradient

# How to overcome this long-term dependency issue?

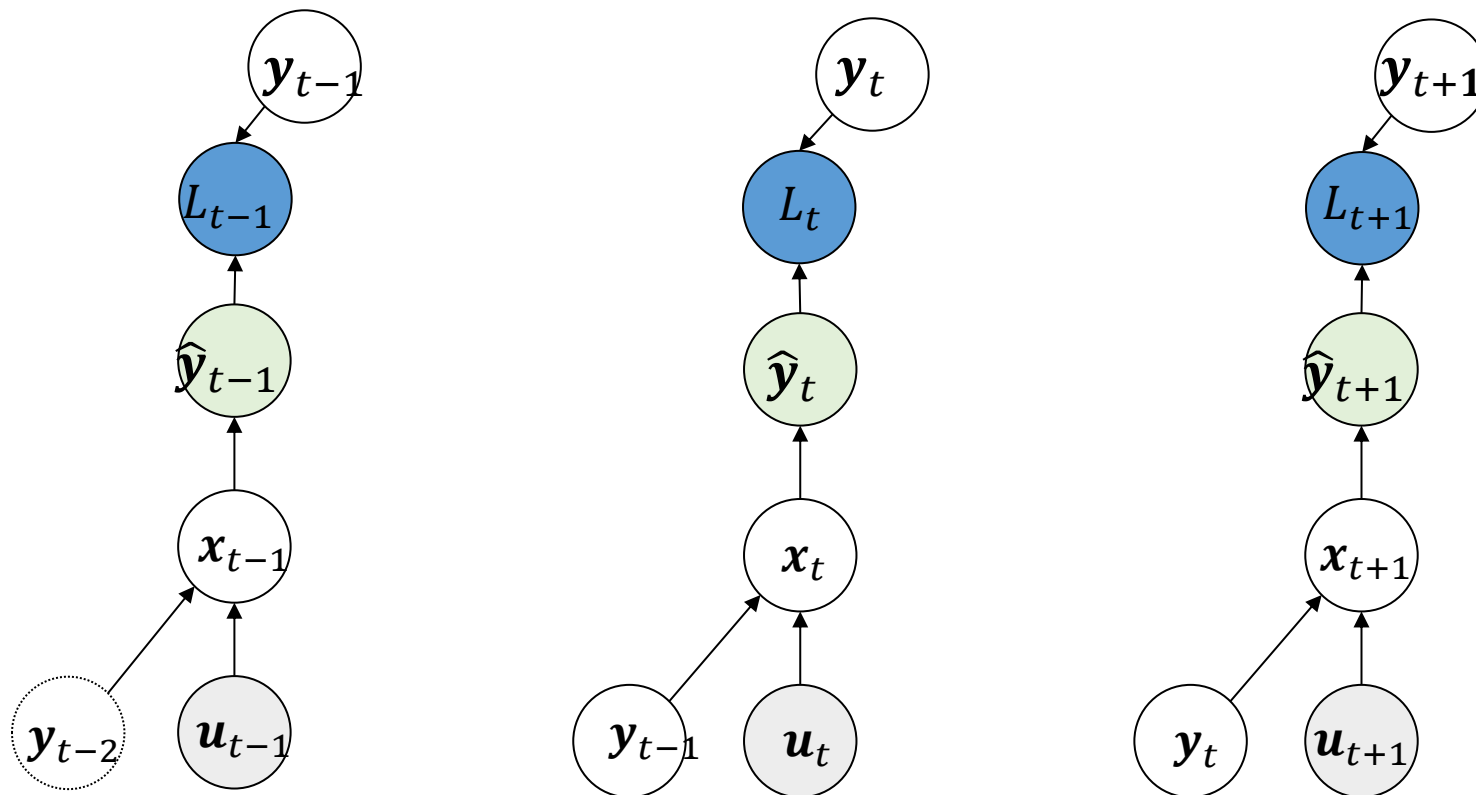
1. Use activation function that don't decrease  $f'$  too much (ReLU)



2. Initialize your Weight matrix  $\mathbf{W}$  such that  $\rho(\mathbf{W}) = 1$  (identity matrix)
3. Use more complex recurrent unit with gates to control the information flow (LSTM)

# Teacher forcing can decouple the graph

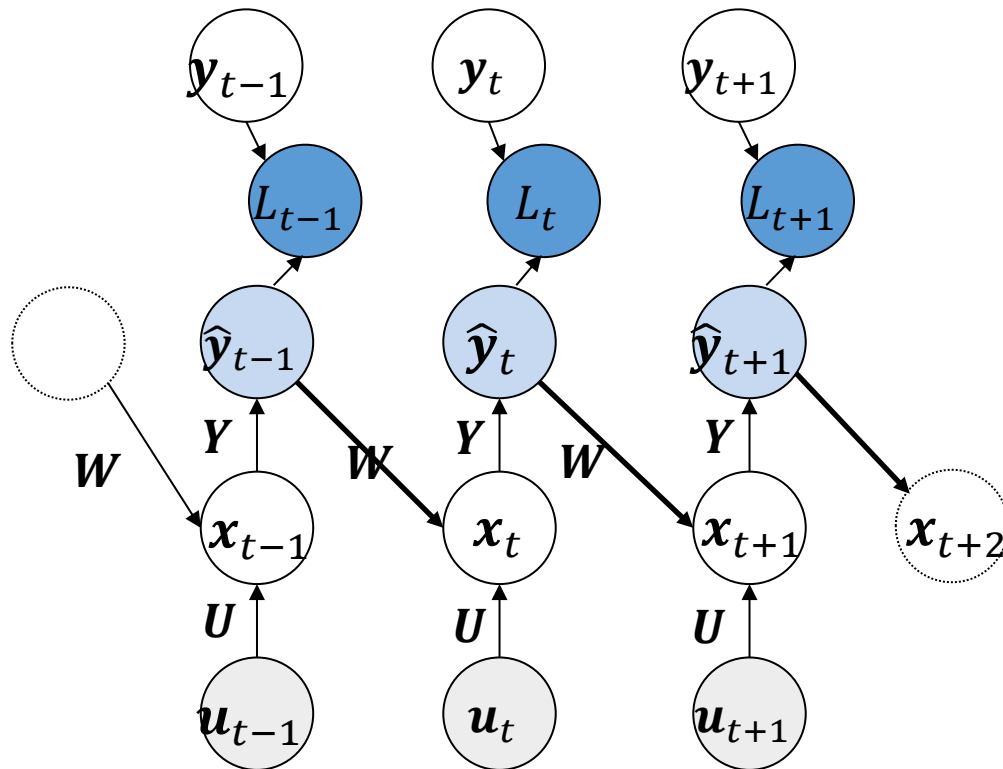
- To facilitate training we can feed the next target output as input and split the graph



! But does not train for long-term prediction !

# Auto-regressive training: Feedback of the prediction back as input

Auto-regressive training:



More expensive training but more robust

# Recurrent Neural Network – Wrap up

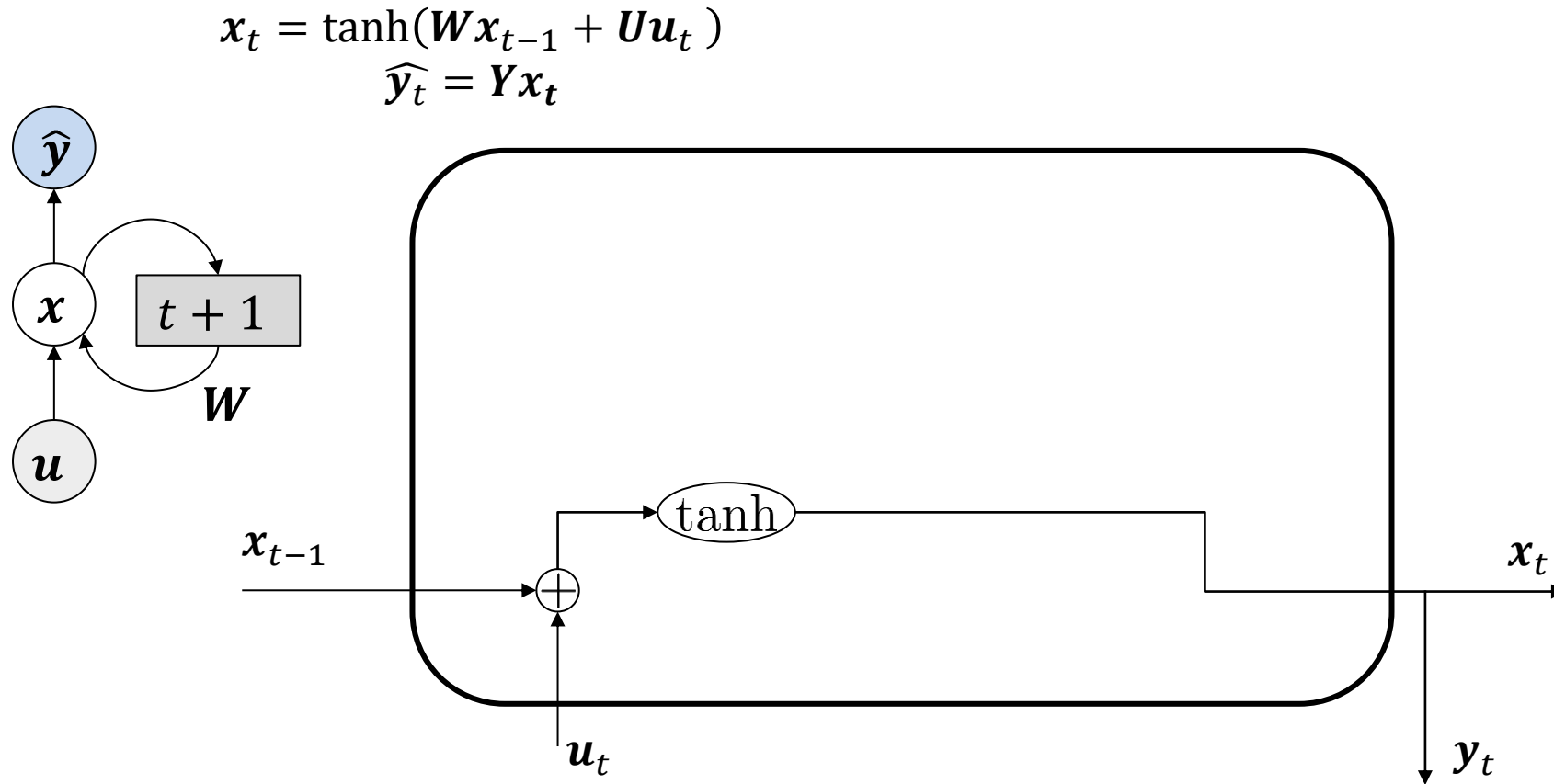
- RNN: Dynamical system with free parameters
  - Similarly to feedforward neural networks, RNN are universal approximators of *dynamical systems*
- Can be trained using Backpropagation Through Time (as usual)
  - But depth of backpropagation depends on number of time-step dependency
  - Teacher-forcing can be used to decouple the graph

# Outline for

## *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
2. Backpropagation through Time
- 3. Long Short-Term Memory Unit**
4. Other RNNs/Advanced RNNs

# Simple RNN



No long-term dependency!

# Long-Short Term Memory

- Idea of not updating the whole hidden state each time
- Protect the state from being overwritten by useless information
- Be selective in
  - What to write (input gate)  $\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_{t-1} + \mathbf{U}_i \mathbf{x}_t + \mathbf{b}_i)$
  - What to read (output gate)  $\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_{t-1} + \mathbf{U}_o \mathbf{x}_t + \mathbf{b}_o)$
  - What to forget (forget gate)  $\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_{t-1} + \mathbf{U}_f \mathbf{x}_t + \mathbf{b}_f)$

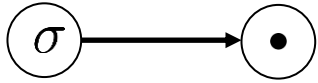
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_x \mathbf{x}_{t-1} + \mathbf{U}_x \mathbf{x}_t + \mathbf{b}_x)$$

$$\mathbf{x}_t = \mathbf{o}_t \odot \mathbf{c}_t$$

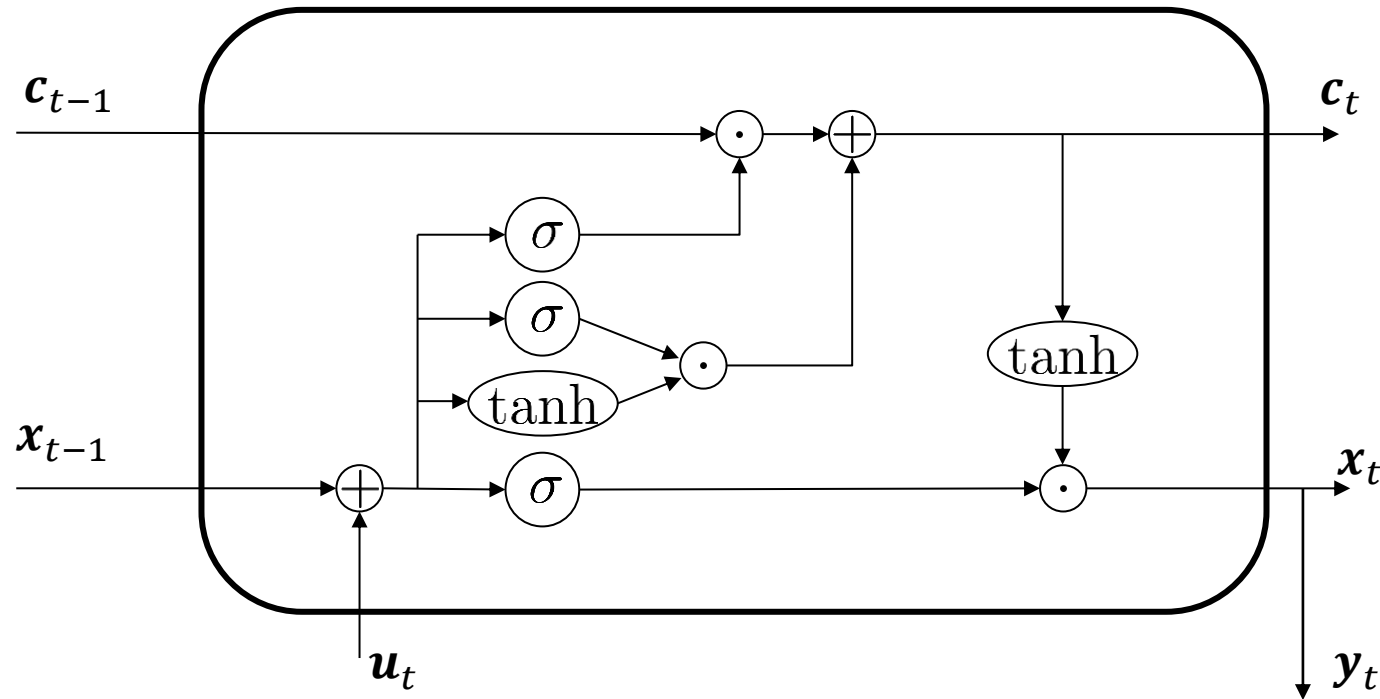


# Long-Short Term Memory Unit

Gate: add/remove information with sigmoid

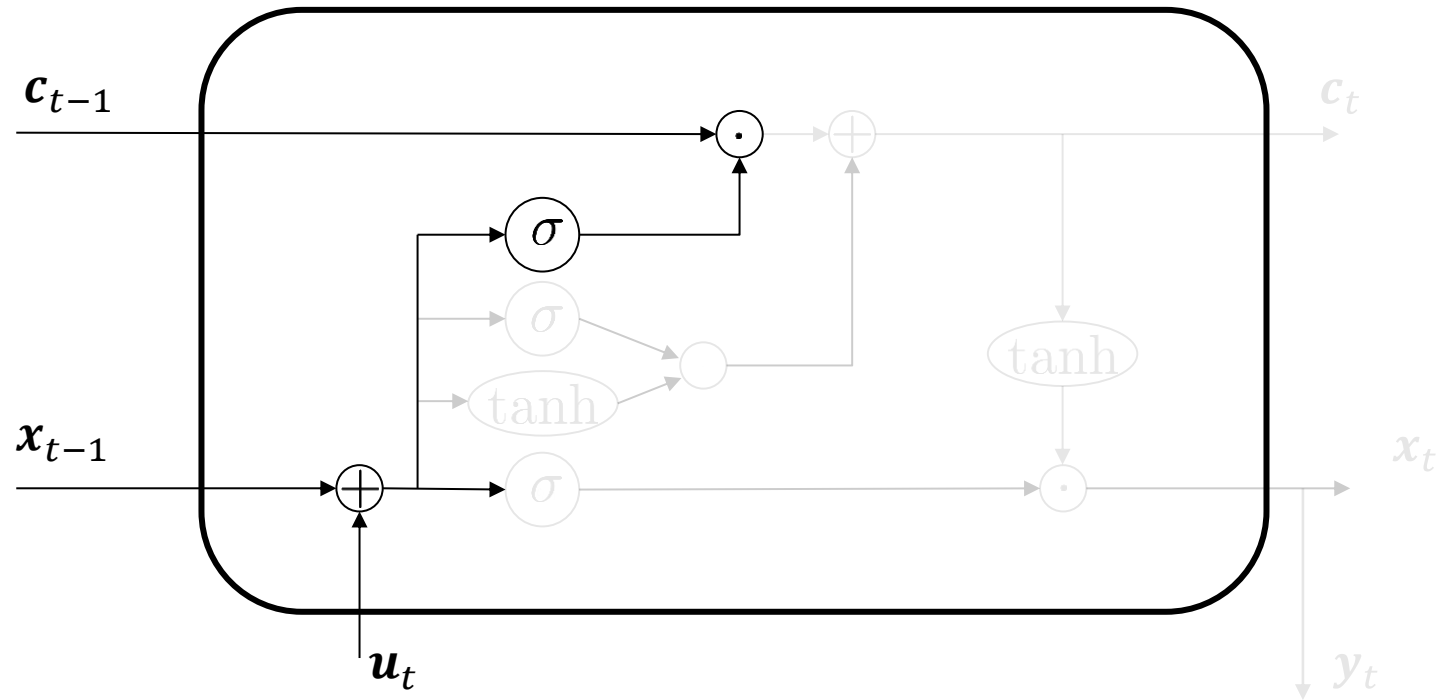


1. Forget
2. Store
3. Update
4. Output



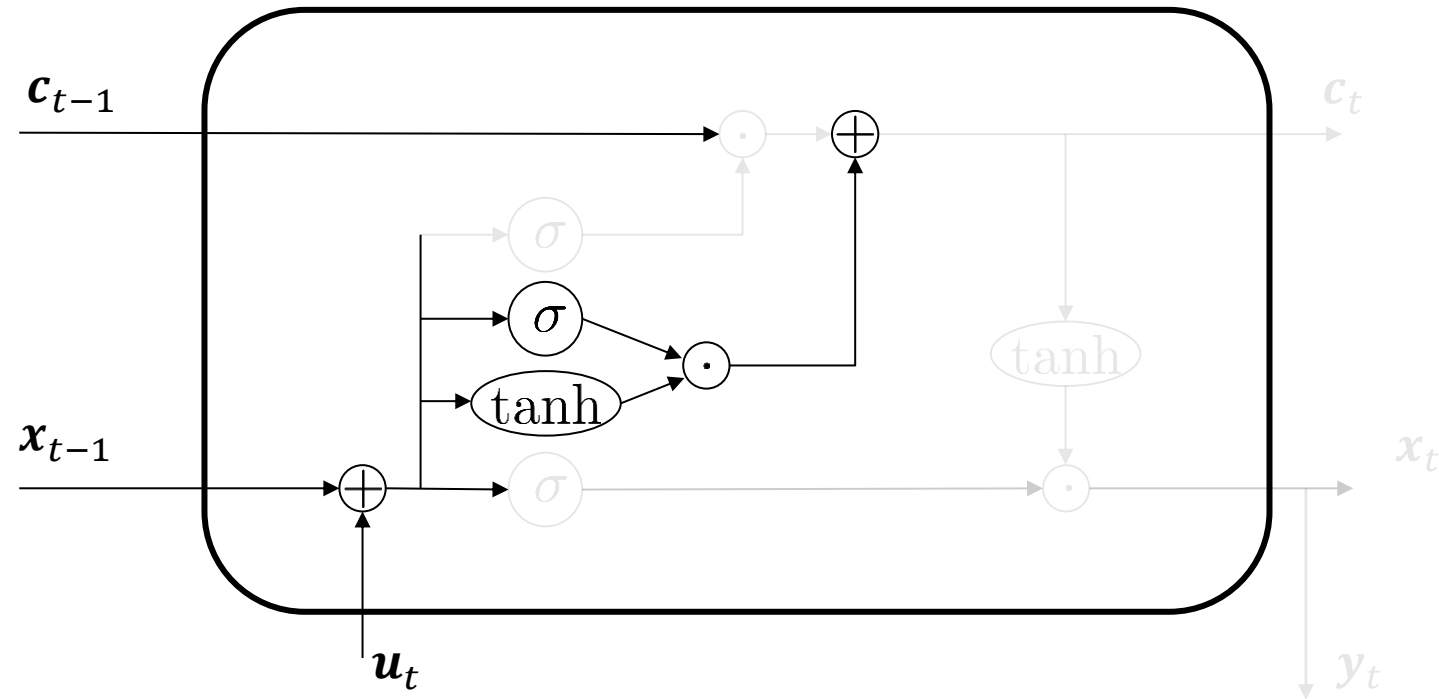
# LSTM - Forget

1. **Forget**
2. Store
3. Update
4. Output



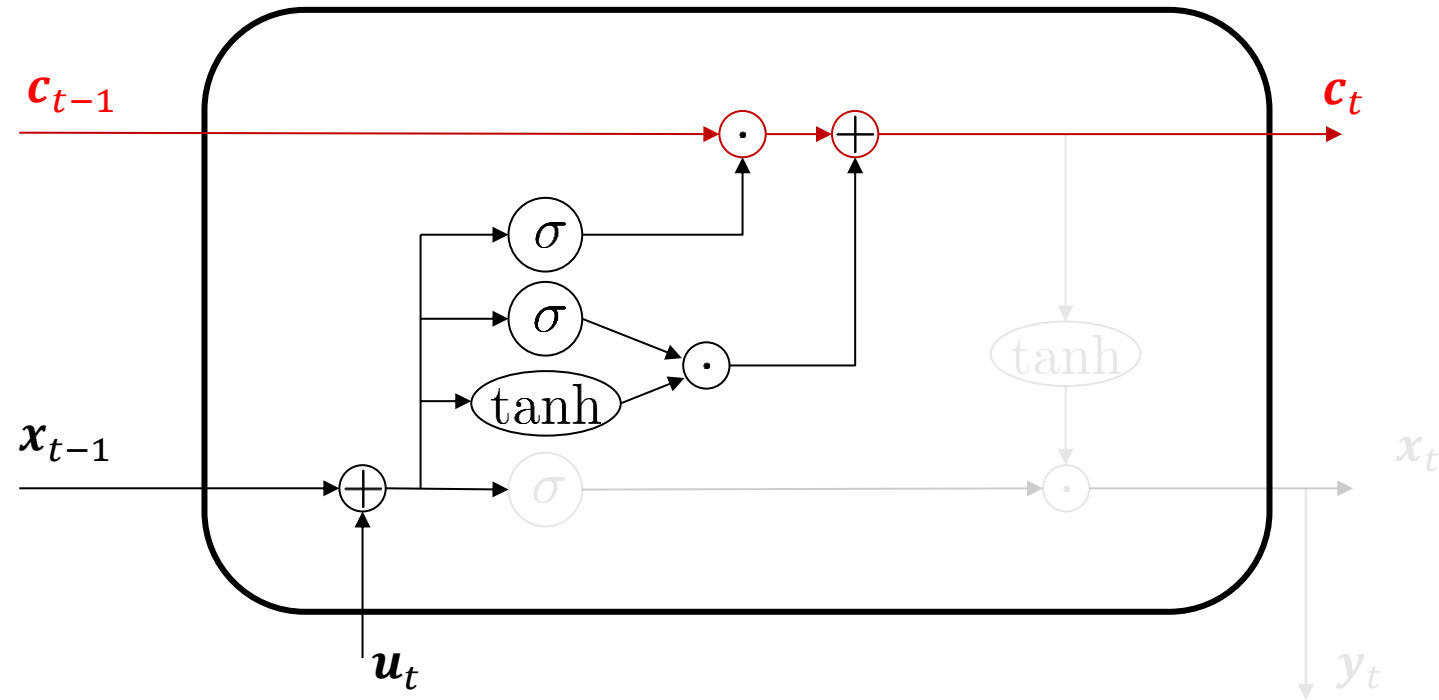
# LSTM – Input (store)

1. Forget
2. **Store**
3. Update
4. Output



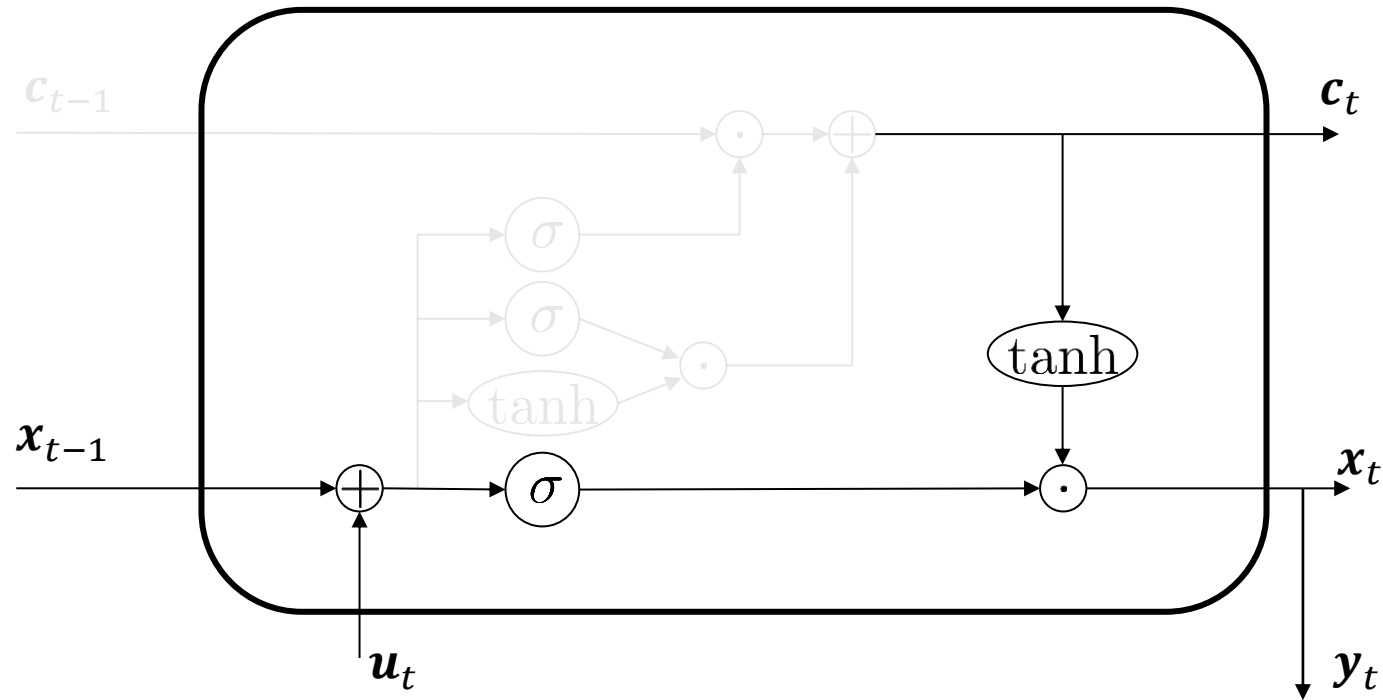
# LSTM – Update

1. Forget
2. Store
- 3. Update**
4. Output

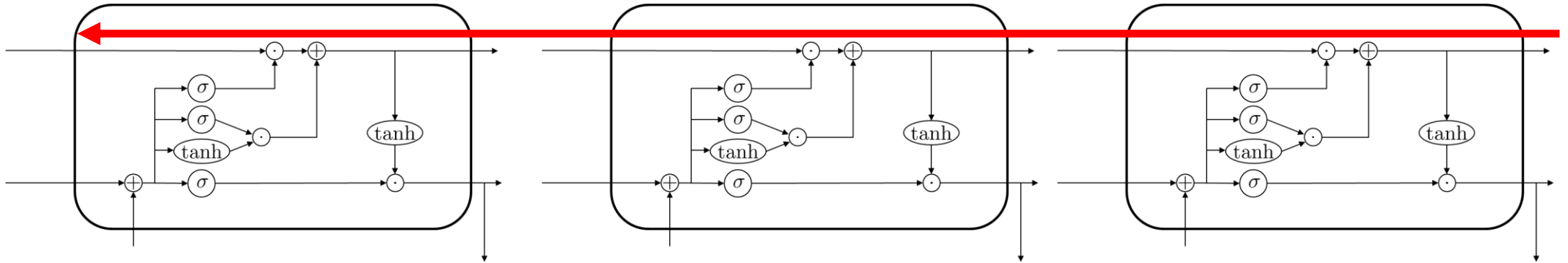


# LSTM - Output

1. Forget
2. Store
3. Update
4. **Output**



# LSTM allows for the uninterrupted gradient calculation



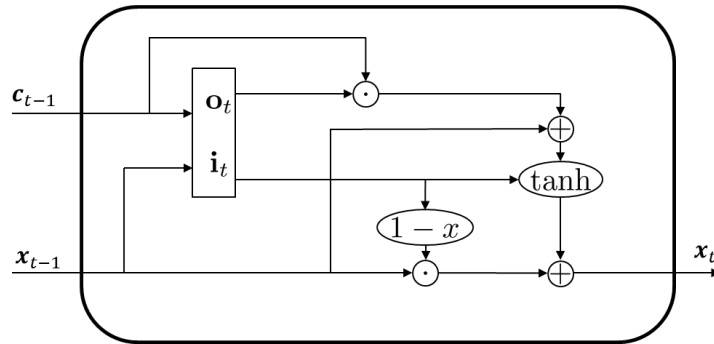
# Outline for

## *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
2. Backpropagation through Time
3. Long Short-Term Memory Unit
- 4. Other RNNs/Advanced RNNs**

# Gated Recurrent Unit

- Alternative to LSTM: GRU (Gated Recurrent Unit)



Input:  $i_t = \sigma(W_i x_{t-1} + U_i x_t + b_i)$

Forget:  $f_t = 1 - i_t$

Output/read:  $o_t = \sigma(W_o x_{t-1} + U_o x_t + b_o)$

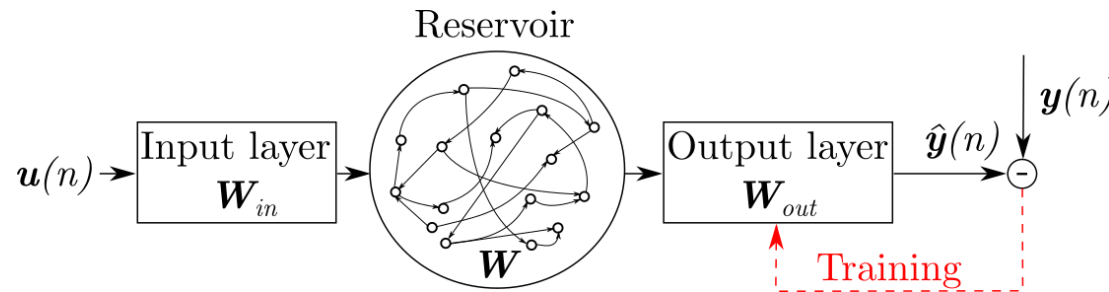
$$x_{t+1} = f_t \odot x_{t-1} + i_t \odot \tanh(W_x x_{t-1} + U_x (o_t \odot c_{t-1}) + b_x)$$

Similar to LSTM



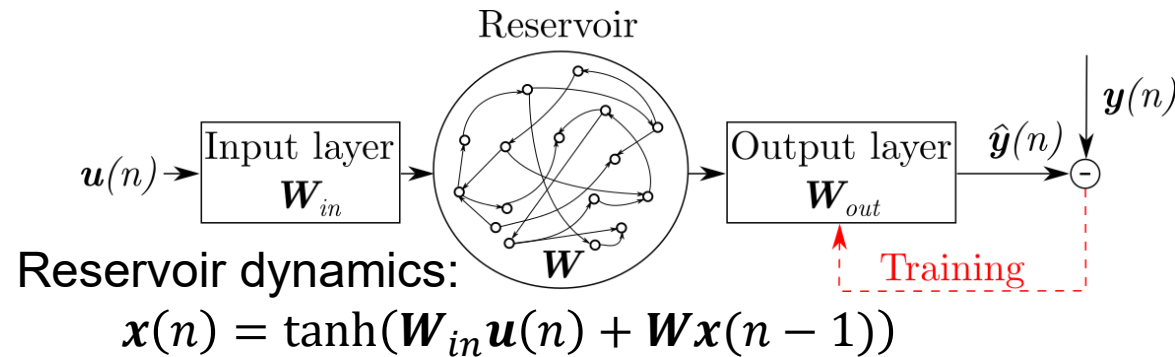
# Reservoir computing is an alternative to LSTM

- Echo State Network:

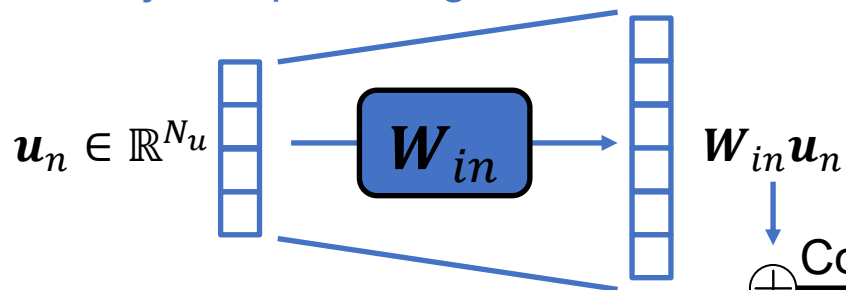


- Static reservoir: high-dimensional fixed random system excited by the input
  - Base for “nonlinear expansion” of the system
  - Memory of the inputs
  - Mapping of the system’s dynamics on the reservoir states via “trained” linear readout
  - No need for backpropagation

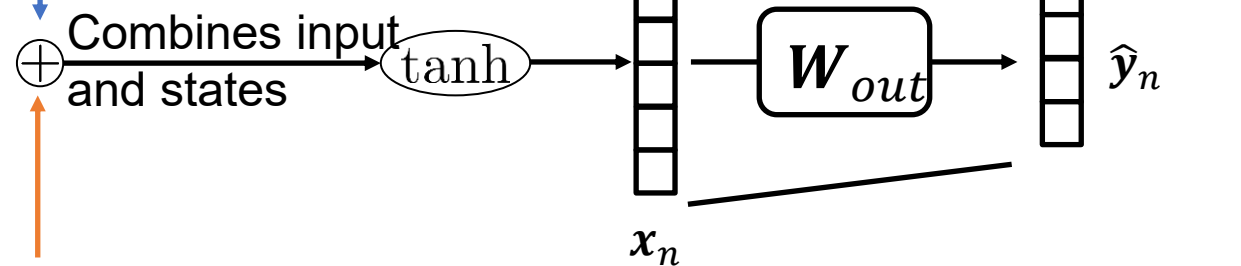
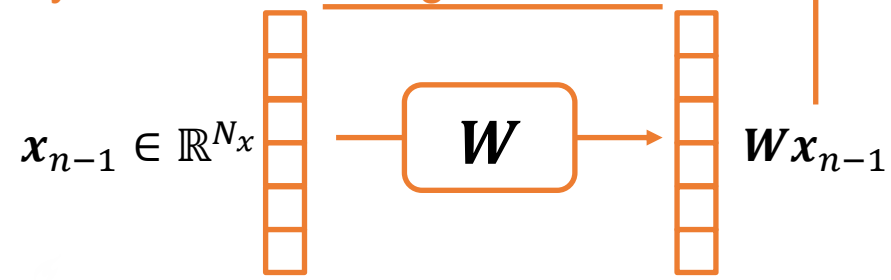
# Reservoir computing is an alternative to LSTM



Project input to higher dimension



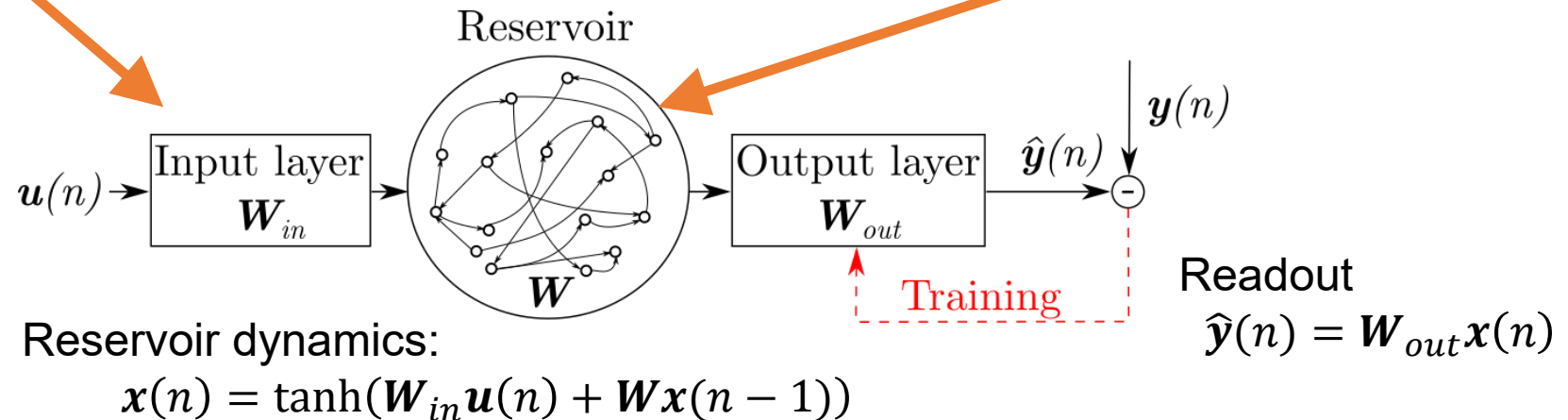
Dynamics in that higher dimension



# Echo State Network

$W_{in}$ : Sparse/Dense matrix randomly initialised from uniform distribution  $[-\sigma_{in}, \sigma_{in}]$

$W$ : Sparse matrix randomly initialised from uniform distribution with set spectral radius  $\Lambda \leq 1$  (echo state property)



Training of  $W_{out}$  based on:

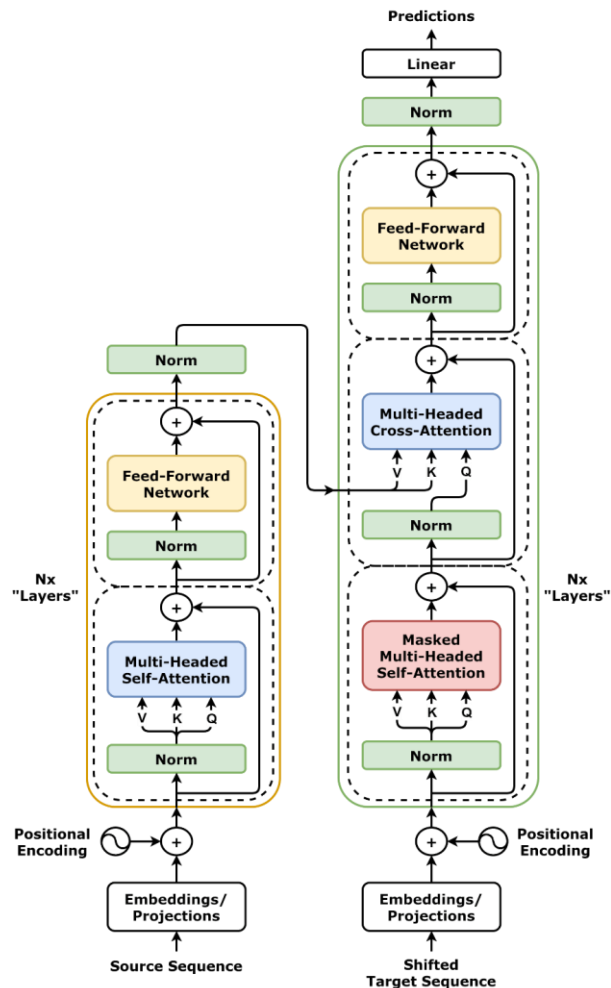
$$L = \underbrace{\frac{1}{N_y} \sum_{i=1}^{N_y} \frac{1}{N_t} \sum_{n=1}^{N_t} (\hat{y}_i(n) - y_i(n))^2}_{\text{Error on data}} + \underbrace{\gamma \frac{1}{N_y} \sum_{i=1}^{N_y} \|\mathbf{w}_{out,i}\|^2}_{\text{Regularity of } \mathbf{W}_{out}}$$

# Training of the ESN is obtained by simple ridge regression

$$L = \frac{1}{N_y} \sum_{i=1}^{N_y} \frac{1}{N_t} \sum_{n=1}^{N_t} (\hat{y}_i(n) - y_i(n))^2 + \gamma \frac{1}{N_y} \sum_{i=1}^{N_y} \|\mathbf{w}_{out,i}\|^2$$

- $\mathbf{W}_{out}$  obtained from ridge regression with
- $\mathbf{W}_{out} = \mathbf{Y}\mathbf{X}^T (\mathbf{X}\mathbf{X}^T + \gamma \mathbf{I})^{-1}$
- Where
- $\mathbf{X}$ : Concatenated reservoir state
- $\mathbf{Y}$ : Concatenated target data
- $\mathbf{I}$ : Identity matrix

# Very brief view on Transformers



Transformers:

- Underpinning most recent Large Language Model

Key features:

- Does not rely on a “recurrence” mechanism
  - So potentially computationally faster
    - Parallelization possible
    - No back propagation through time
- Relies on “Positional Embedding” and “Attention mechanism”

# Very brief view on Transformers

## Encoder - Positional encoding

Assume a sequence  $x(n)$  of length  $N$  with  $x \in \mathbb{R}^d \rightarrow X \in \mathbb{R}^{N \times d}$

Positional encoding matrix  $P \in \mathbb{R}^{d \times N}$  defined as:

$$P_{i,2j} = \sin\left(\frac{i}{M^{2j/d}}\right)$$

$$P_{i,2j+1} = \cos\left(\frac{i}{M^{2j/d}}\right)$$

Where  $M$  is a very large number ( $M \gg N, d$ ).

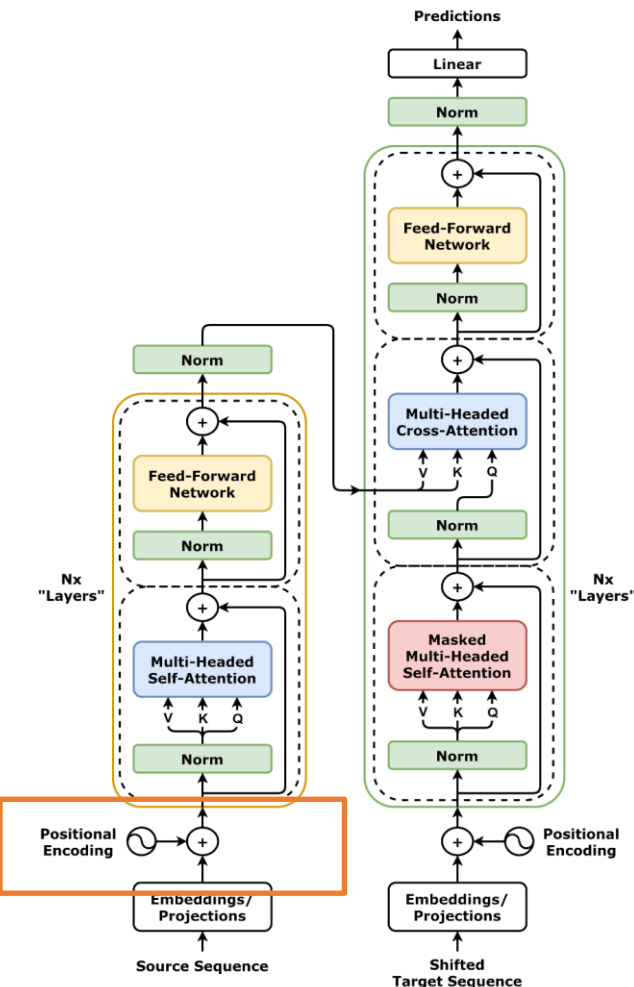
The updated input with positional encoding is:

$$X + P$$

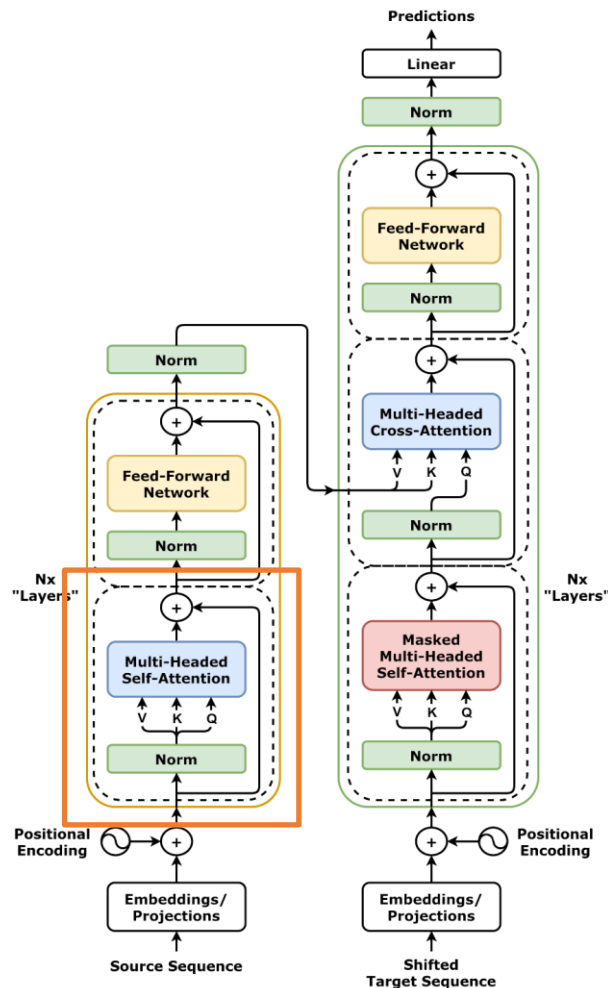
Characteristics of this positional encoding: Shift by offset  $\delta$  equivalent to linear matrix multiplication:

$$\begin{pmatrix} P_{i+\delta,2j} \\ P_{i+\delta,2j+1} \end{pmatrix} = \begin{pmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{pmatrix} \begin{pmatrix} P_{i,2j} \\ P_{i,2j+1} \end{pmatrix}$$

→ Provides mean to identify relative position of element in the sequence



# Very brief view on Transformers



## Encoder - Self-Attention mechanism

Assume a triplet of “(query, key, values)” ( $Q, K, V$ )

- Query: what you’re asking
- Keys: the way the underlying knowledge is indexed
- Values: the set of values associated with the keys

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

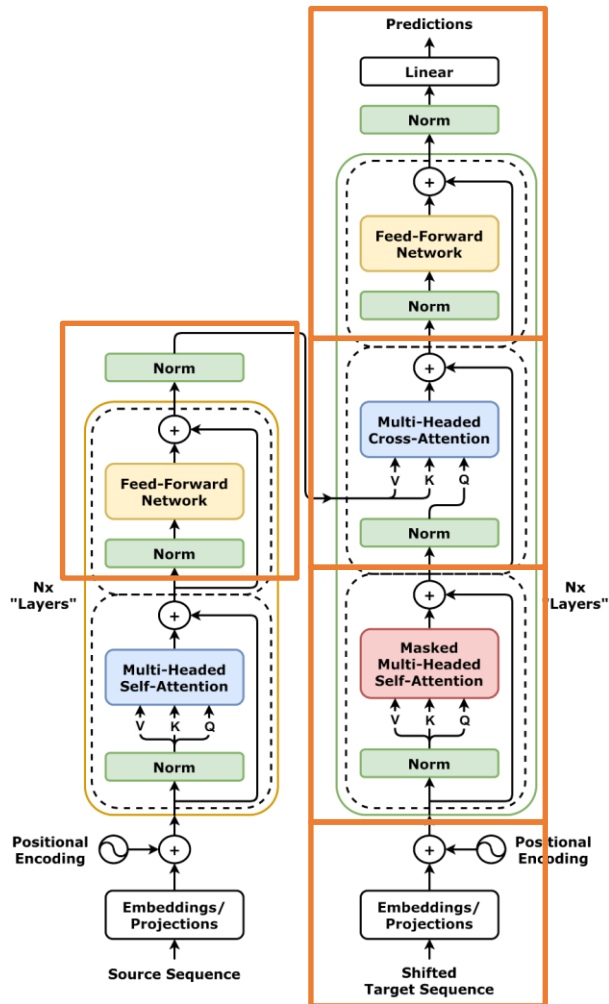
1.  $QK^T$ : “Similarity of your query with respect to the keys”  
→ Similar to a correlation between  $Q$  and  $K$
2. Softmax: provides a probability associated to the values
3. Multiplied by the value (to be predicted)  $V$

In practice for self-attention:  $Q = K = V \equiv X$

All matrices multiplied by weight matrices (to be learned)

$$\text{multiheadattention} = \text{concat}[\text{Attention}(W_q Q, W_k K, W_v V)]$$

# Very brief view on Transformers



## Encoder - Feedforward network

- “Combine” the predicted values and feed to cross-attention (in decoded)

## Decoder – Positional Embedding/Self-Attention

- Positional embedding: applied on the target sequence
- Masked self-attention: similar as on the encoder-side but masking to ensure “causality” (i.e. no cross correlation with future attention)

## Decoder – Cross-Attention

- Combine the self-attention (from target sequence) with transformed attention (from encoder)

$$\text{multiheadattention} = \text{concat}[\text{Attention}(W_q^{Dec} Q^{Dec}, W_k^{Enc} K^{enc}, W_v^{Enc} V^{Enc})]$$

## Decoder – Feedforward/Linear

- Final transformation for prediction



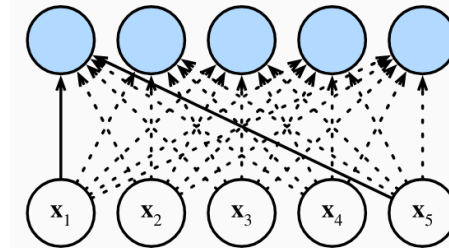
# Very brief view on Transformers

## Core mechanism for “sequence handling”

- Attention-mechanism:

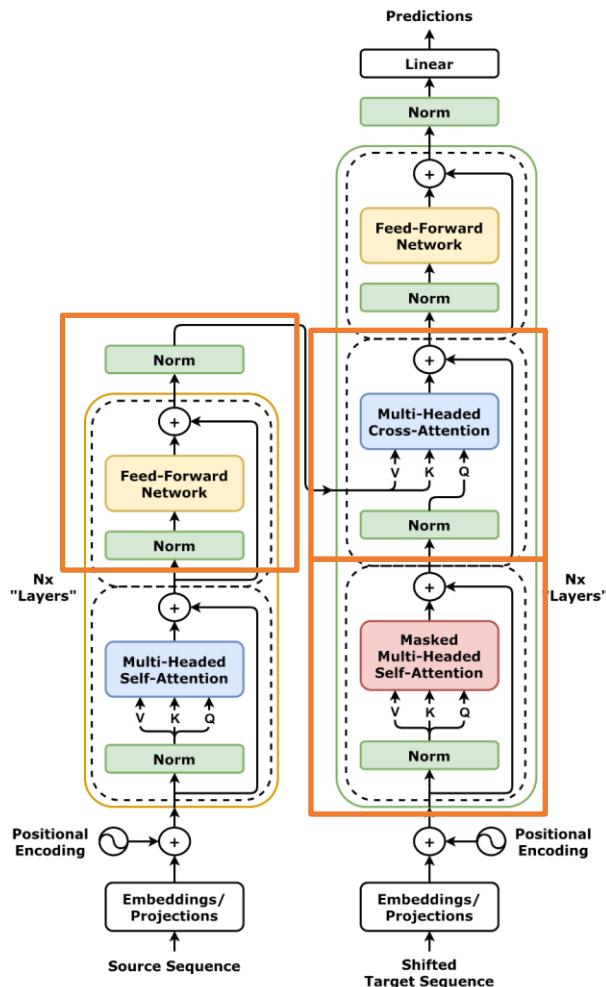
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

→  $\mathbf{Q}\mathbf{K}^T$ : parallelizable – does not require recurrence



Similar to a fully-connected NN approach

- Time-dependency: dependent on embedding length and dimension of  $\mathbf{Q}$  (query length accepted)
- Can be very memory intensive
- But allow for potentially more efficient training through parallelization

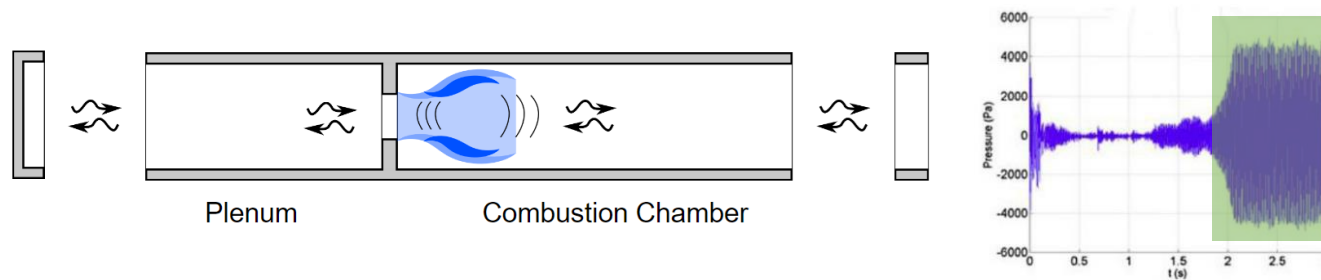


# Outline

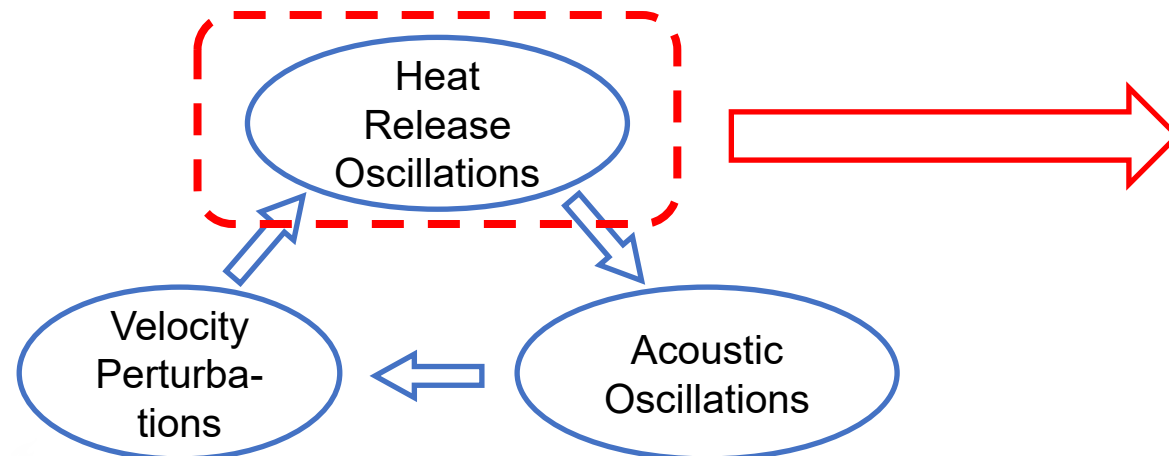
- Key concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and Dynamical Systems
  - System Identification for LTI system
  - Data-driven nonlinear dynamical system modelling
- **Applications to reacting flows**
  - Flame dynamics modelling
  - Reduced-order modelling

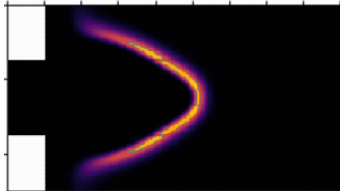
# Typical dynamical modelling in turbulent (reacting) flows: Flow dynamics modelling

- Thermoacoustic instabilities: High amplitude pressure oscillations in premixed combustors



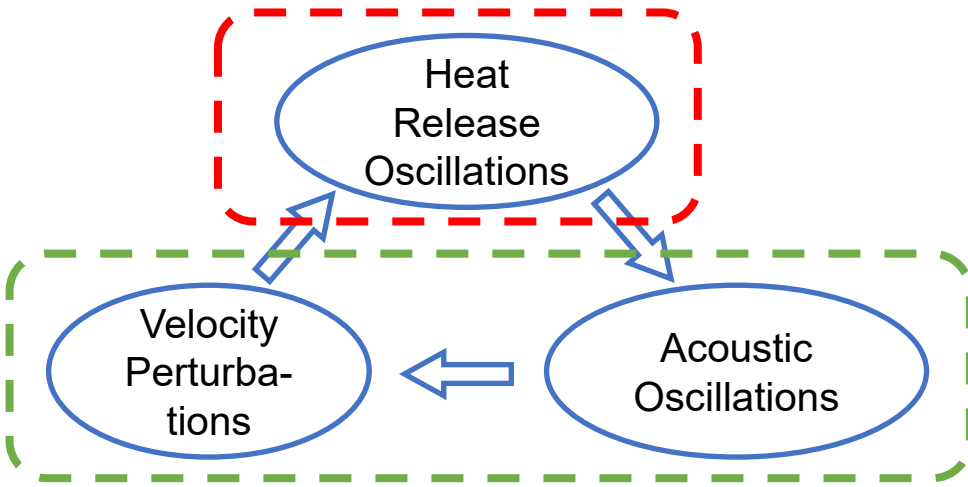
Thermoacoustic instabilities: result from coupling between HRR and acoustic fluctuations



$$\dot{\phi} = N(\phi, p, F)$$


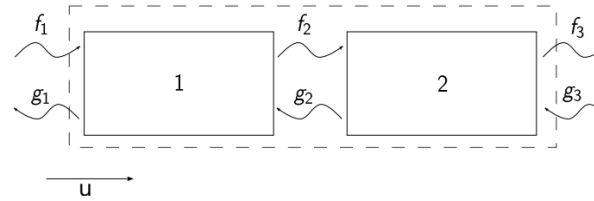
$$\rightarrow \dot{Q} = \mathcal{F}(\phi, F)$$

# Divide and Conquer approach relies on interacting flame and acoustic models

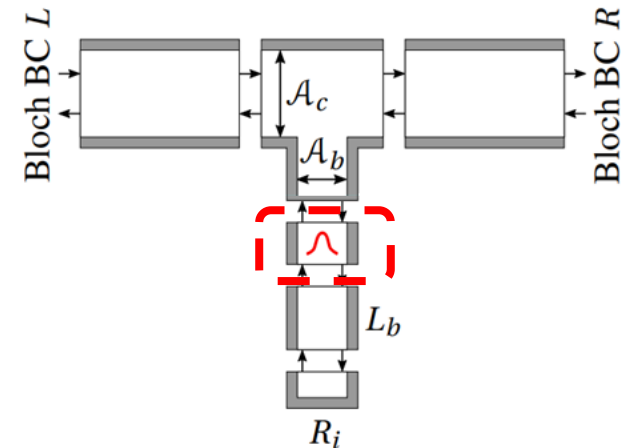
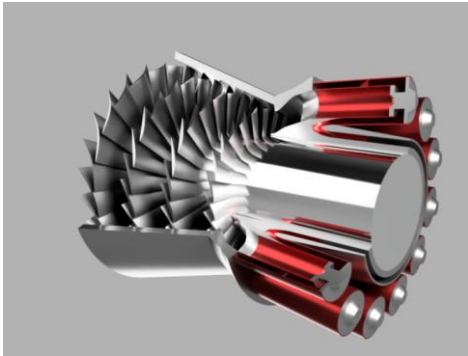


Flame model:  $\dot{Q}' = \mathcal{F}(u')$

Acoustic network model



$$\begin{aligned} \dot{x}_1 &= A_1 x_1 + B_1 \begin{bmatrix} f_1 \\ g_2 \end{bmatrix} & y_1 &= \begin{bmatrix} g_1 \\ f_2 \end{bmatrix} \\ \dot{x}_2 &= A_2 x_2 + B_2 \begin{bmatrix} f_2 \\ g_3 \end{bmatrix} & y_2 &= \begin{bmatrix} g_2 \\ f_3 \end{bmatrix} \end{aligned}$$

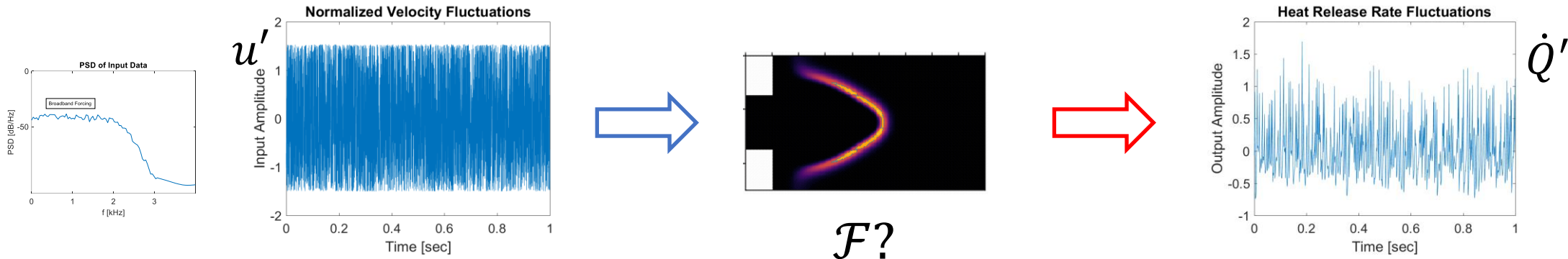


# Essential of the flame dynamics can be obtained with single broadband forcing simulation

- What data do we need to model the flame dynamics?

$$\dot{Q}' = \mathcal{F}(u')$$

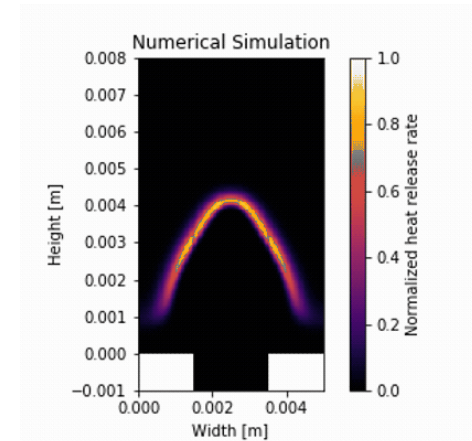
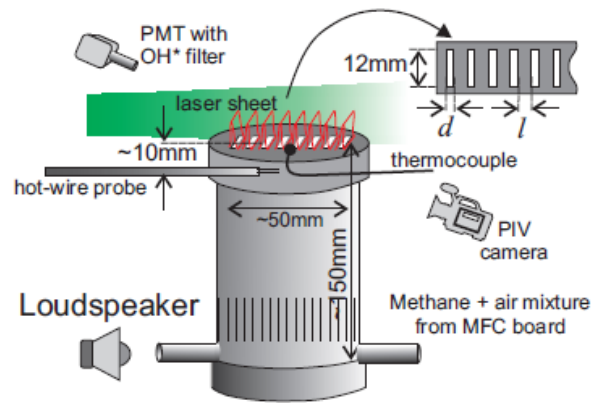
- From system identification:
  - Dataset with broadband forcing contains all the necessary information



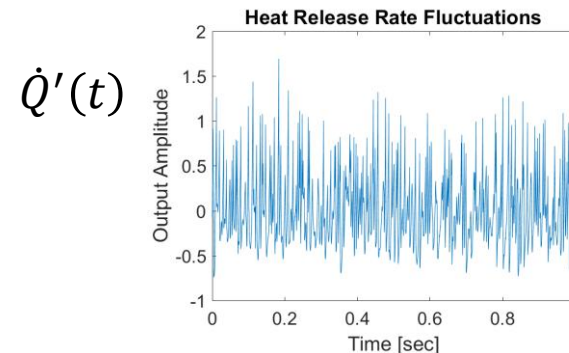
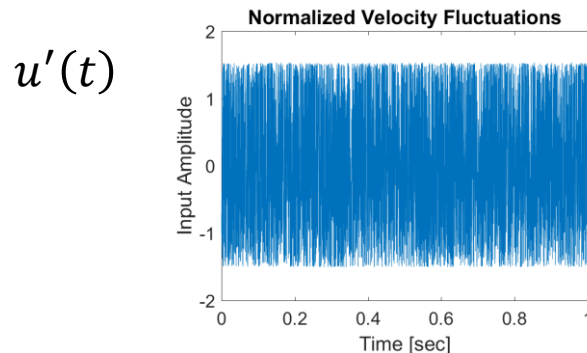
- How to model this?
  - Typical supervised learning problem applied to sequential data

# Test case is a laminar premixed methane/air flame

- Fully resolved lean premixed methane-air slit burner at  $\phi = 0.8$

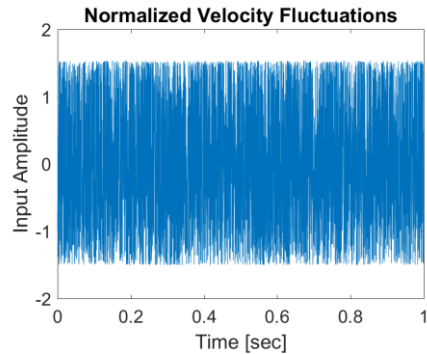


- Dataset: One simulation broadband excitation at high amplitude



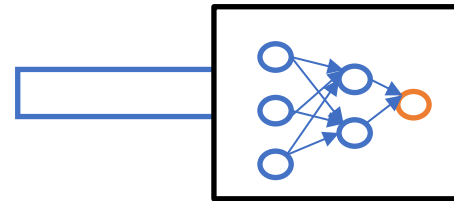
# Neural networks can accurately learn nonlinear flame dynamics

NN trained with one broadband simulation at high amplitude

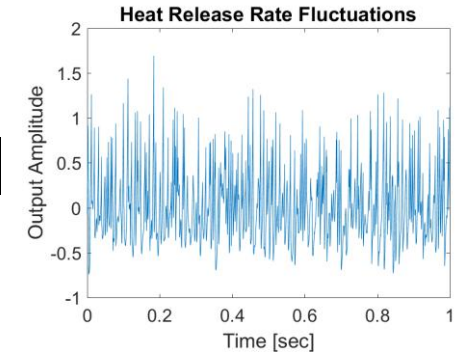


$$\begin{bmatrix} u'(t) \\ u'(t - \Delta t) \\ \vdots \\ u'(t - n\Delta t) \end{bmatrix}$$

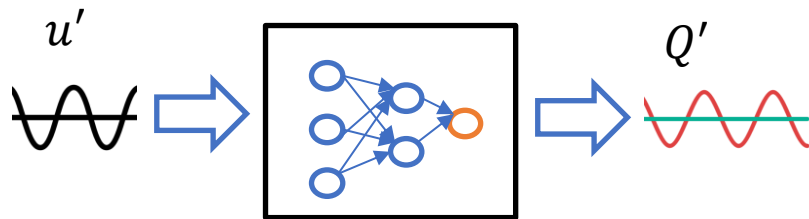
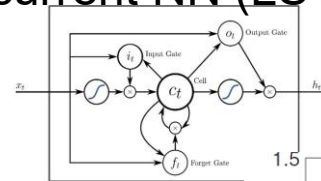
Feedforward NN



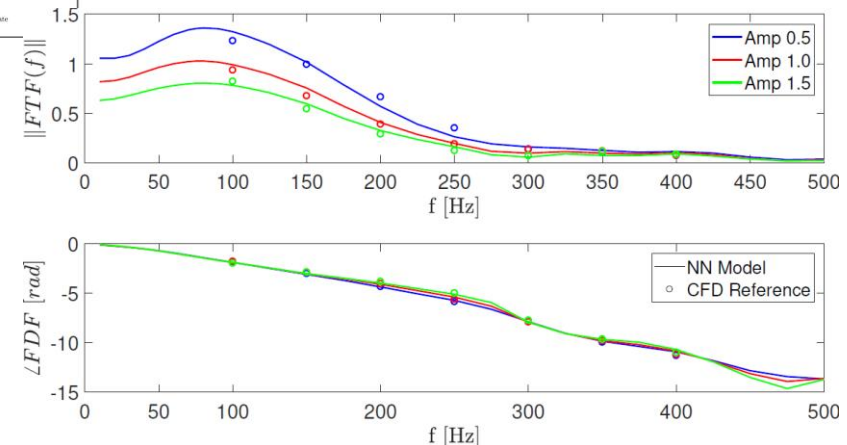
$$[\dot{Q}'(t)]$$



Recurrent NN (LSTM)



Frequency response of trained network

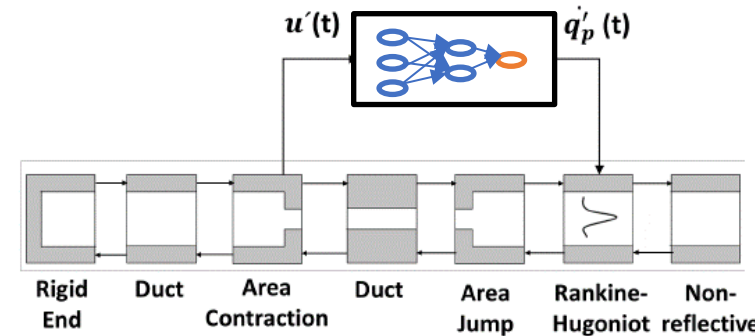
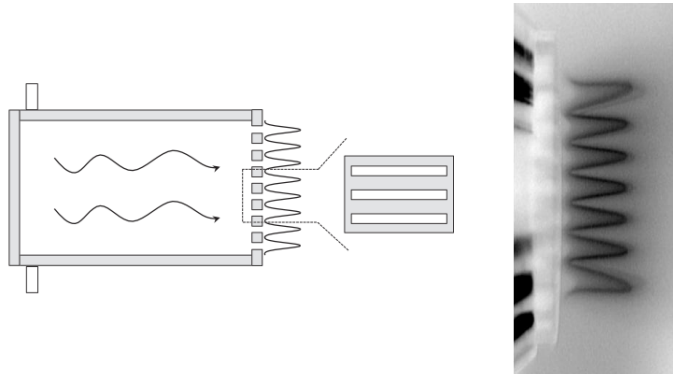


[Modeling of the nonlinear flame response of a Bunsen-type flame via multi-layer perceptron, Tathawadekar et al., Proc. Comb. Inst. (38), 2021]  
[Physics-informed recurrent neural networks for linear and nonlinear flame dynamics, Yadav et al., Proc. Comb. Inst. (39), 2023]

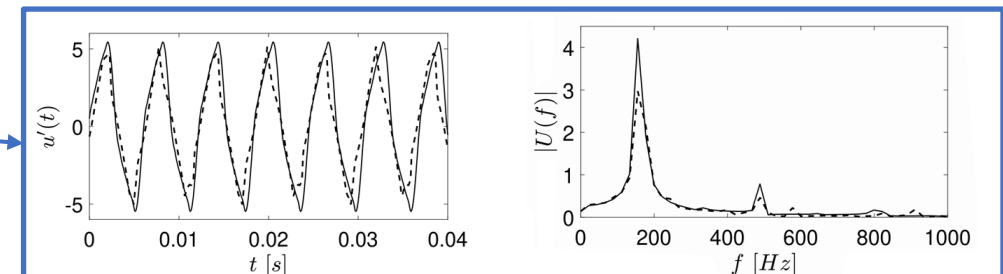
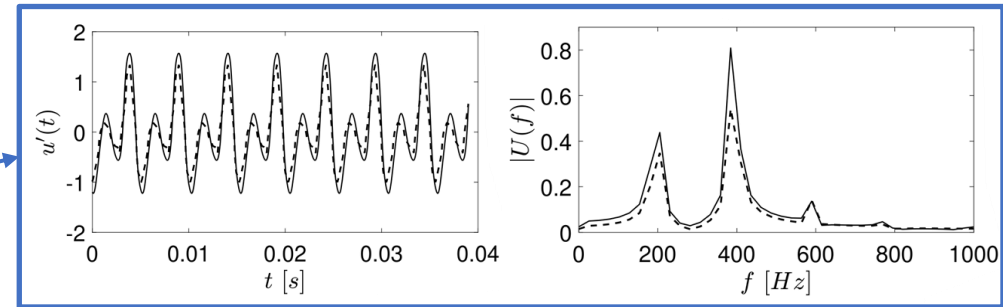
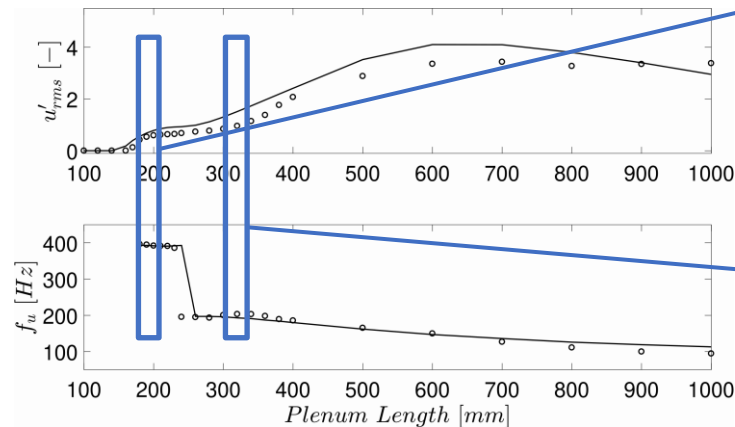


# Integrated NN and acoustic network predicts accurately the thermoacoustic instability

- Trained NN with acoustic model describes the multi-slit combustor



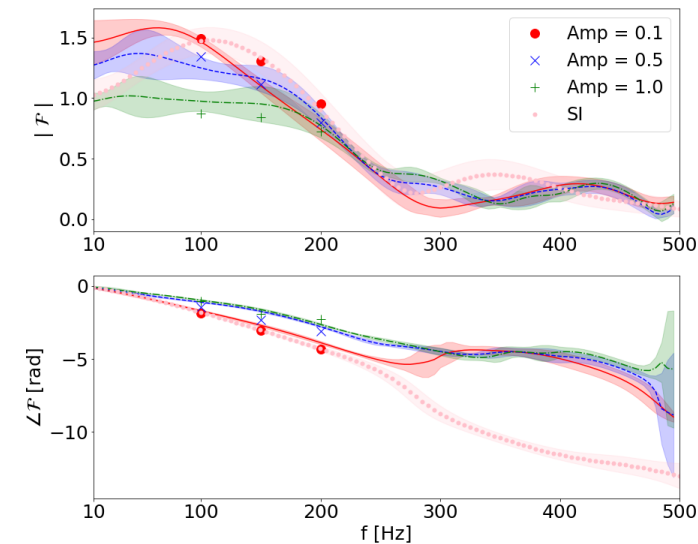
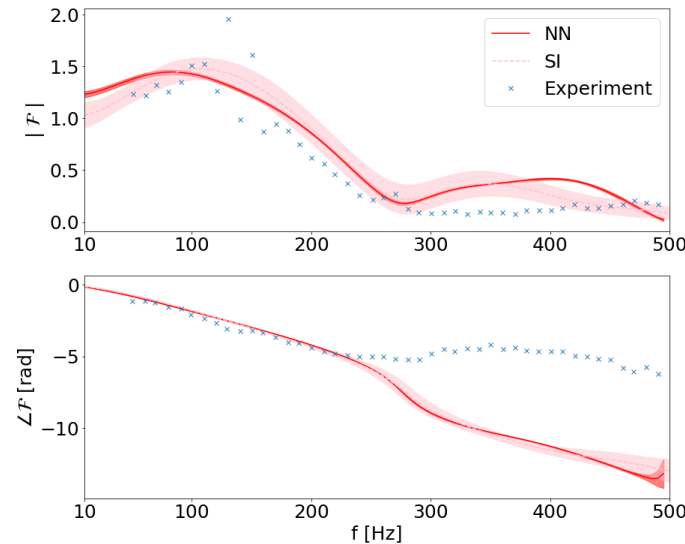
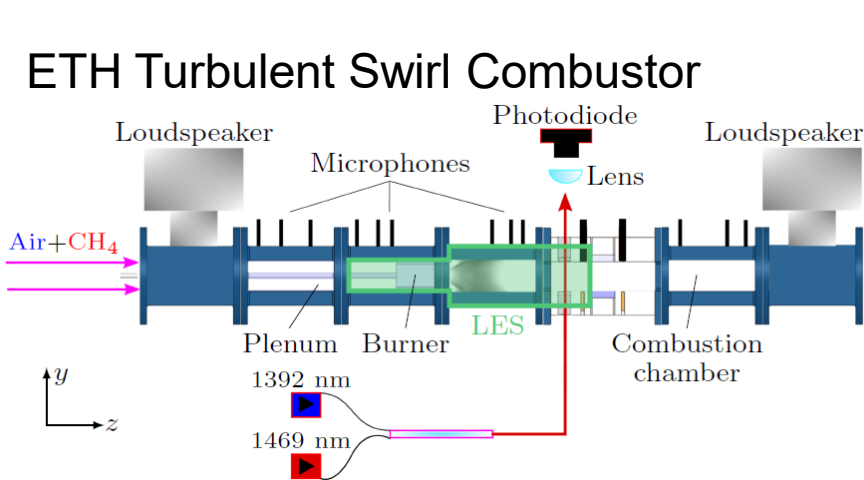
## Design stability analysis





# Extension to turbulent flames

- Previous methodology also applicable to turbulent flames



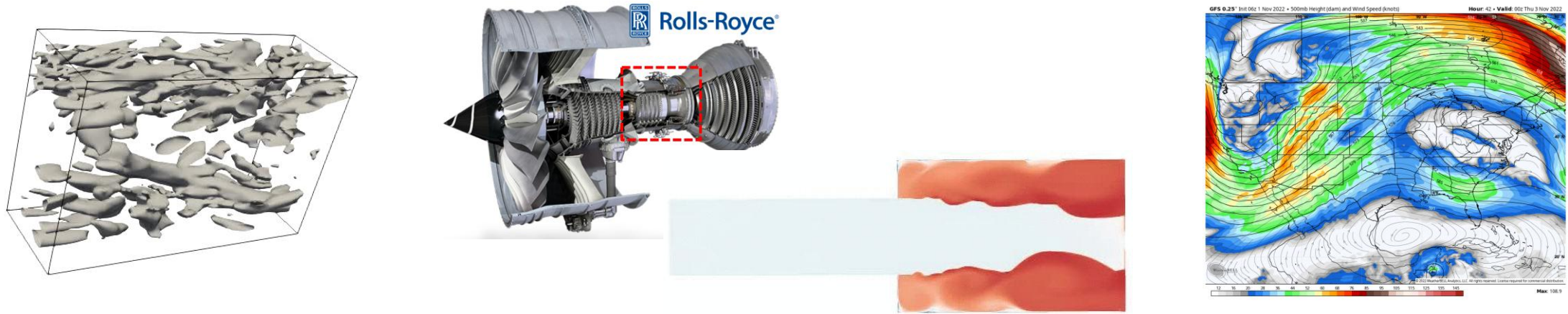
Increased challenge due to turbulent fluctuations

# Outline

- Key concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and Dynamical Systems
  - System Identification for LTI system
  - Data-driven nonlinear dynamical system modelling
- **Applications to reacting flows**
  - Flame dynamics modelling
  - **Reduced-order modelling**

# Learning turbulent dynamics of high-dimensional flows

- Most turbulent flows are highly dimensional



How can we handle those with all the methods seen so far?

→ Split the tasks

Spatial structures

→ Handled with CNN

(form of nonlinear modal decomposition)

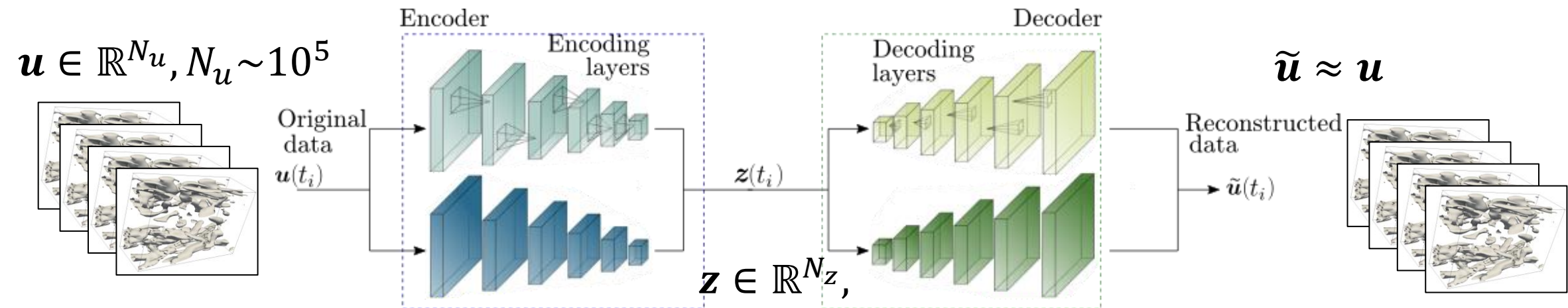
Temporal chaotic dynamics

→ Handled with RNN

→ In essence similar to a POD-Galerkin ROM approach

# The spatial structures can be identified with CNN-based networks

- CNN-based autoencoder: discovers efficient latent space
  - Based on identifying “spatial structures”
  - Encoder: finds an efficient reduced order representation (latent space)
  - Decoder: reconstructs the full flow state from the latent space

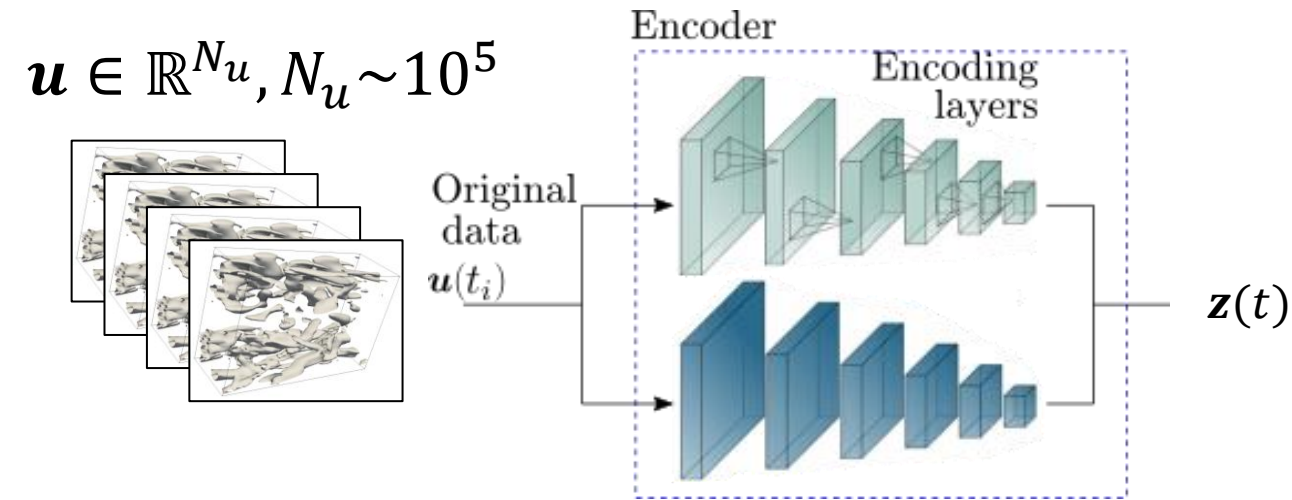


Trained by minimizing: 
$$J = \frac{1}{N_t} \sum_i \|u^{(i)} - \tilde{u}^{(i)}\|^2$$

[Predicting turbulent dynamics with the convolutional autoencoder echo state network, Racca et al., J. Fluid Mech. (975), 2023]

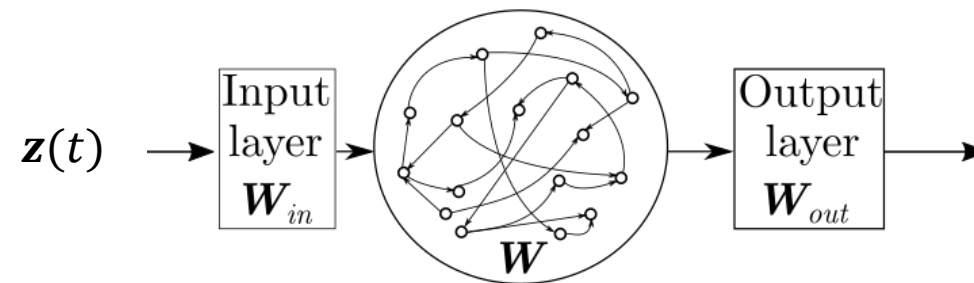
[Convolutional autoencoded echo state network for the prediction of extreme events in turbulence, Doan et al., CTRSP-2022, 2022] 79

# The encoder provides flow dynamics in the latent space



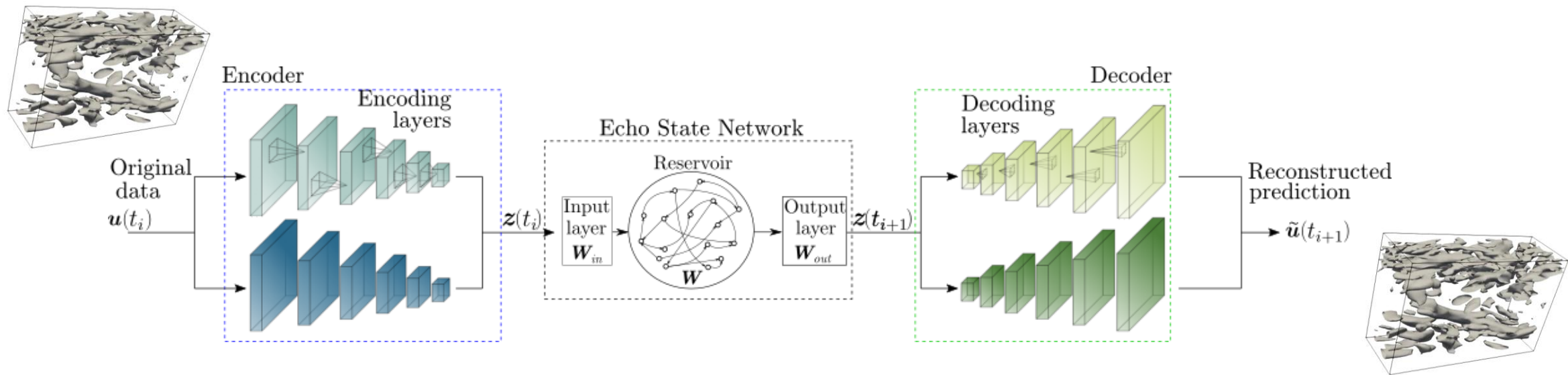
- “Encoded” version of original dataset
- Can be used to learn the dynamics
- Decoder can be used to recover the full physical state

We can learn that dynamics (in the latent space) with an RNN



# By combining CNN and RNN, we can handle high-dimensional flows

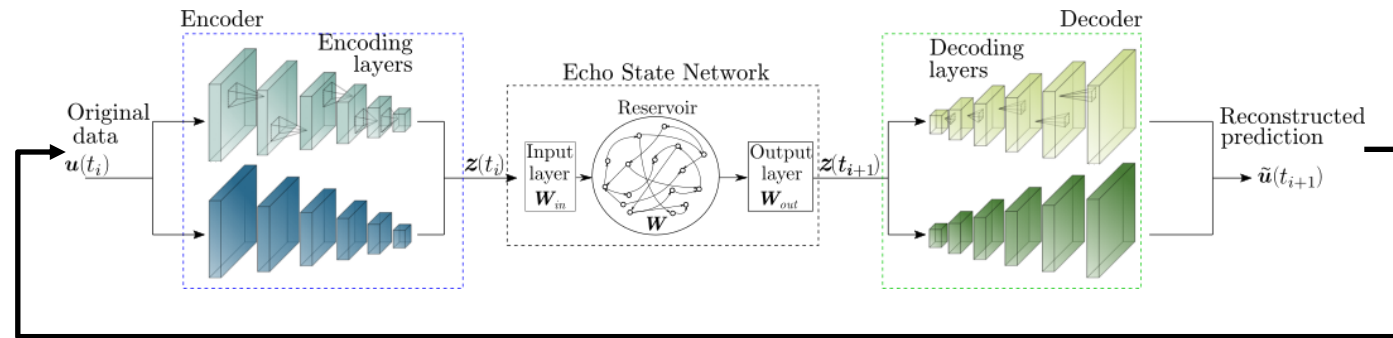
- Overall architecture:



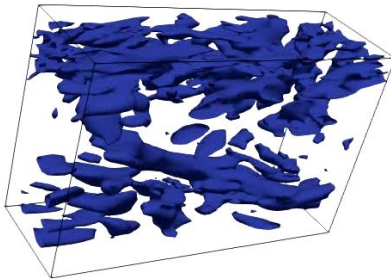


# The hybrid model can accurately reproduce the flow dynamics

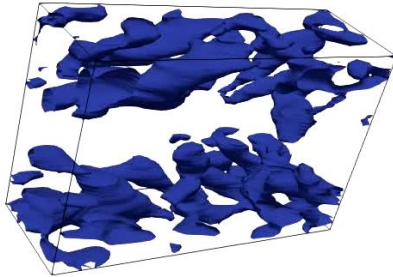
After training



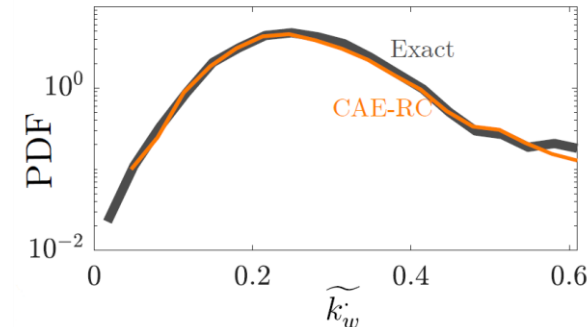
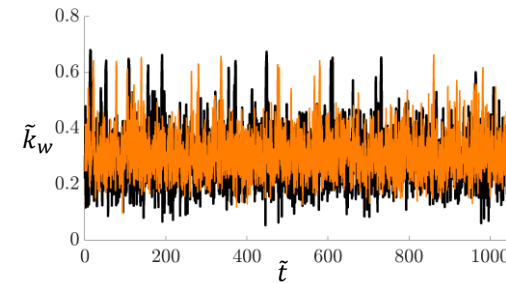
Actual



ML model

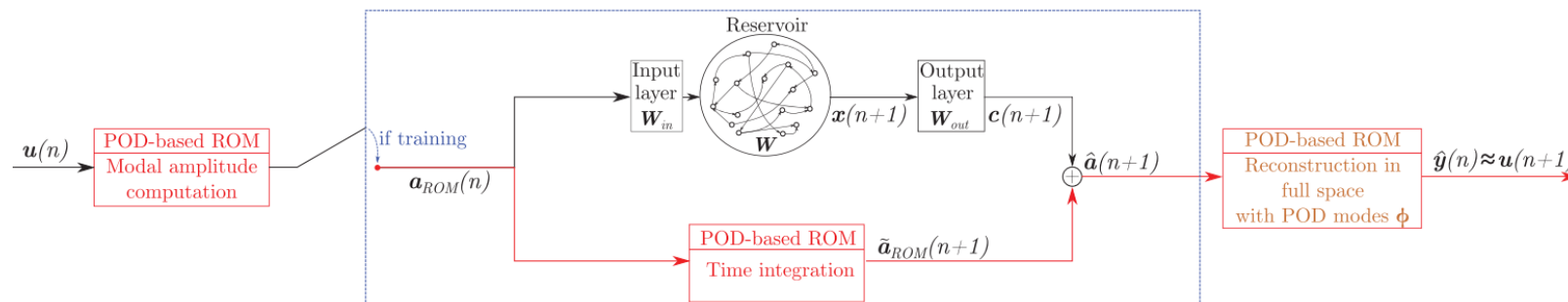


Extreme events statistics



# Summary

- Chaotic systems are difficult to predict
  - Comparison based on: statistics/attractors and short-term predictions
- Sequential data requires particular processing
- Modelling of dynamical system
  - If LTI system: Impulse response describes the entire dynamics
  - If nonlinear:
    - Families of Recurrent Neural Networks exists
- Note: Combination of methods possible





# Questions?

[n.doan@imperial.ac.uk](mailto:n.doan@imperial.ac.uk)